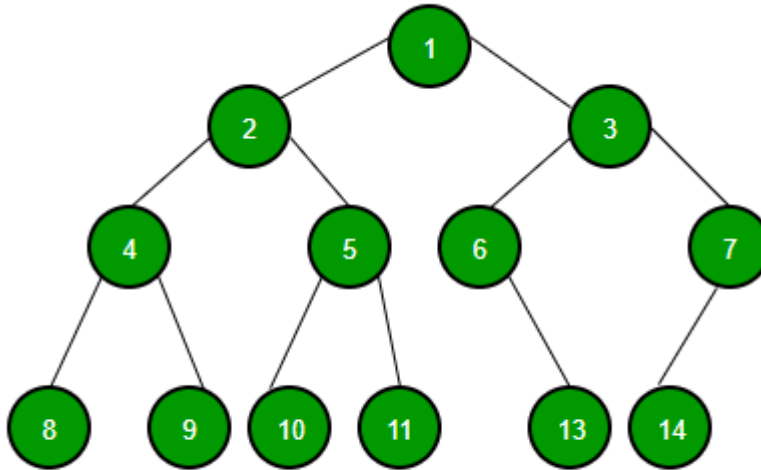


UNIT – IV

Binary Trees

What is Binary Tree Data Structure?

Binary Tree is defined as a tree data structure where each node has at most 2 children. Since each element in a binary tree can have only 2 children, we typically name them the left and right child.



Binary Tree Representation

A Binary tree is represented by a pointer to the topmost node (commonly known as the “root”) of the tree. If the tree is empty, then the value of the root is NULL. Each node of a Binary Tree contains the following parts:

Data

Pointer to left child

Pointer to right child

Basic Operation On Binary Tree:

Inserting an element.

Removing an element.

Searching for an element.

Traversing the tree.

Auxiliary Operation On Binary Tree:

Finding the height of the tree

Find the level of a node of the tree

Finding the size of the entire tree.

```
// Class containing left and right child
// of current node and key value
class Node {
    int key;
    Node left, right;

    public Node(int item)
    {
        key = item;
        left = right = null;
    }
}
```

Example: Java Program to Implement Binary Tree

```
// class to create nodes
class Node {
    int key;
    Node left, right;

    public Node(int item) {
        key = item;
        left = right = null;
    }
}

class BinaryTree {
    Node root;

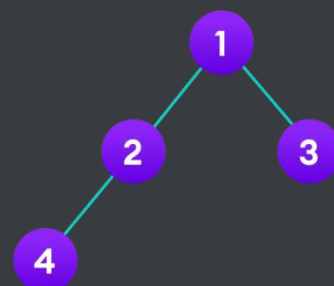
    // Traverse tree
    public void traverseTree(Node node) {
        if (node != null) {
            traverseTree(node.left);
            System.out.print(" " + node.key);
            traverseTree(node.right);
        }
    }

    public static void main(String[] args) {

        // create an object of BinaryTree
        BinaryTree tree = new BinaryTree();

        // create nodes of the tree
        tree.root = new Node(1);
        tree.root.left = new Node(2);
        tree.root.right = new Node(3);
        tree.root.left.left = new Node(4);

        System.out.print("\nBinary Tree: ");
        tree.traverseTree(tree.root);
    }
}
```



Output: 4 2 1 3

Tree traversal (Inorder, Preorder and Postorder)

In this article, we will discuss the tree traversal in the data structure. The term 'tree traversal' means traversing or visiting each node of a tree. There is a single way to traverse the linear data structure such as linked list, queue, and stack. Whereas, there are multiple ways to traverse a tree that are listed as follows -

- Preorder traversal
- Inorder traversal
- Postorder traversal

So, in this article, we will discuss the above-listed techniques of traversing a tree. Now, let's start discussing the ways of tree traversal.

Preorder traversal

This technique follows the 'root left right' policy. It means that, first root node is visited after that the left subtree is traversed recursively, and finally, right subtree is recursively traversed. As the root node is traversed before (or pre) the left and right subtree, it is called preorder traversal.

So, in a preorder traversal, each node is visited before both of its subtrees.

The applications of preorder traversal include -

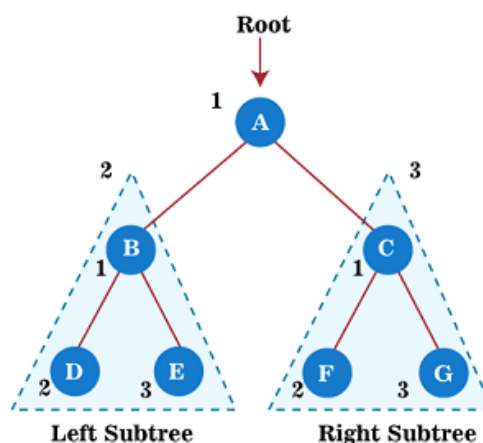
- It is used to create a copy of the tree.
- It can also be used to get the prefix expression of an expression tree.

Algorithm

1. Until all nodes of the tree are not visited
2. Step 1 - Visit the root node
3. Step 2 - Traverse the left subtree recursively.
4. Step 3 - Traverse the right subtree recursively.

Example

Now, let's see the example of the preorder traversal technique.



Now, start applying the preorder traversal on the above tree. First, we traverse the root node **A**; after that, move to its left subtree **B**, which will also be traversed in preorder.

So, for left subtree B, first, the root node **B** is traversed itself; after that, its left subtree **D** is traversed. Since node **D** does not have any children, move to right subtree **E**. As node E also does not have any children, the traversal of the left subtree of root node A is completed.

Now, move towards the right subtree of root node A that is C. So, for right subtree C, first the root node **C** has traversed itself; after that, its left subtree **F** is traversed. Since node **F** does not have any children, move to the right subtree **G**. As node G also does not have any children, traversal of the right subtree of root node A is completed.

Therefore, all the nodes of the tree are traversed. So, the output of the preorder traversal of the above tree is -

A → B → D → E → C → F → G

```
// Java program for preorder traversals

class Node {
    int data;
    Node left, right;

    public Node(int item) {
        data = item;
        left = right = null;
    }
}

class BinaryTree {
    Node root;

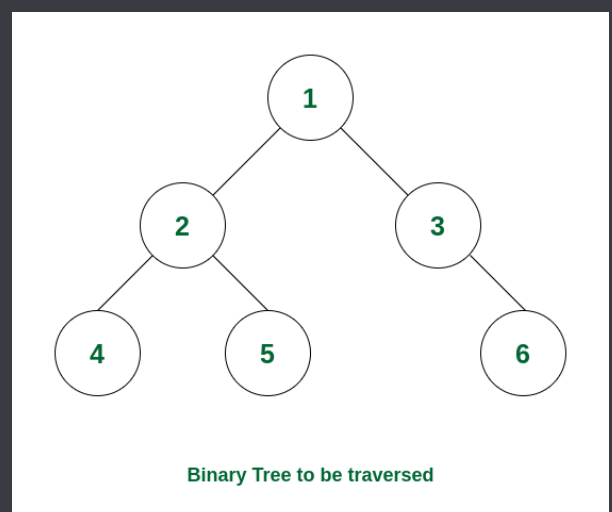
    BinaryTree() {
        root = null;
    }

    // Function to print preorder traversal
    void printPreorder(Node node) {
        if (node == null)
            return;

        // Deal with the node
        System.out.print(node.data + " ");

        // Recur on left subtree
        printPreorder(node.left);

        // Recur on right subtree
        printPreorder(node.right);
    }
}
```



```

// Driver code
public static void main(String[] args) {
    BinaryTree tree = new BinaryTree();

    // Constructing the binary tree
    tree.root = new Node(1);
    tree.root.left = new Node(2);
    tree.root.right = new Node(3);
    tree.root.left.left = new Node(4);
    tree.root.left.right = new Node(5);
    tree.root.right.right = new Node(6);

    // Function call
    System.out.println("Preorder traversal of binary tree is: ");
    tree.printPreorder(tree.root);
}
}

```

Postorder traversal

This technique follows the 'left-right root' policy. It means that the first left subtree of the root node is traversed, after that recursively traverses the right subtree, and finally, the root node is traversed. As the root node is traversed after (or post) the left and right subtree, it is called postorder traversal.

So, in a postorder traversal, each node is visited after both of its subtrees. The applications of postorder traversal include -

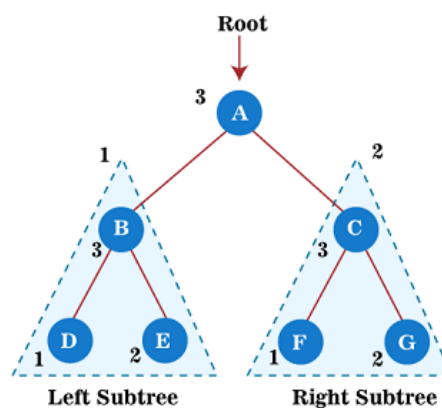
- It is used to delete the tree.
- It can also be used to get the postfix expression of an expression tree.

Algorithm

1. Until all nodes of the tree are not visited
2. Step 1 - Traverse the left subtree recursively.
3. Step 2 - Traverse the right subtree recursively.
4. Step 3 - Visit the root node.

Example

Now, let's see the example of the postorder traversal technique.



Now, start applying the postorder traversal on the above tree. First, we traverse the left subtree **B** that will be traversed in postorder. After that, we will traverse the right subtree **C** in postorder. And finally, the root node of the above tree, i.e., **A**, is traversed.

So, for left subtree **B**, first, its left subtree **D** is traversed. Since node **D** does not have any children, traverse the right subtree **E**. As node **E** also does not have any children, move to the root node **B**. After traversing node **B**, the traversal of the left subtree of root node **A** is completed.

Now, move towards the right subtree of root node **A** that is **C**. So, for right subtree **C**, first its left subtree **F** is traversed. Since node **F** does not have any children, traverse the right subtree **G**. As node **G** also does not have any children, therefore, finally, the root node of the right subtree, i.e., **C**, is traversed. The traversal of the right subtree of root node **A** is completed.

At last, traverse the root node of a given tree, i.e., **A**. After traversing the root node, the postorder traversal of the given tree is completed.

Therefore, all the nodes of the tree are traversed. So, the output of the postorder traversal of the above tree is -

D → E → B → F → G → C → A

```
// Java program for postorder traversals
import java.util.*;

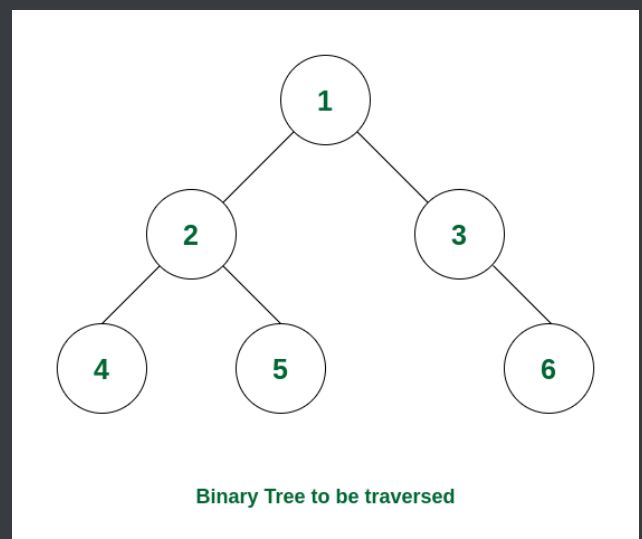
// Structure of a Binary Tree Node
class Node {
    int data;
    Node left, right;
    Node(int v)
    {
        data = v;
        left = right = null;
    }
}

class GFG {

    // Function to print postorder traversal
    static void printPostorder(Node node)
    {
        if (node == null)
            return;

        // First recur on left subtree
        printPostorder(node.left);

        // Then recur on right subtree
        printPostorder(node.right);
```



```

// Now deal with the node
System.out.print(node.data + " ");
}

// Driver code
public static void main(String[] args)
{
    Node root = new Node(1);
    root.left = new Node(2);
    root.right = new Node(3);
    root.left.left = new Node(4);
    root.left.right = new Node(5);
    root.right.right = new Node(6);

    // Function call
    System.out.println("Postorder traversal of binary tree is: ");
    printPostorder(root);
}
}

```

Inorder traversal

This technique follows the 'left root right' policy. It means that first left subtree is visited after that root node is traversed, and finally, the right subtree is traversed. As the root node is traversed between the left and right subtree, it is named inorder traversal.

So, in the inorder traversal, each node is visited in between of its subtrees.

The applications of Inorder traversal includes -

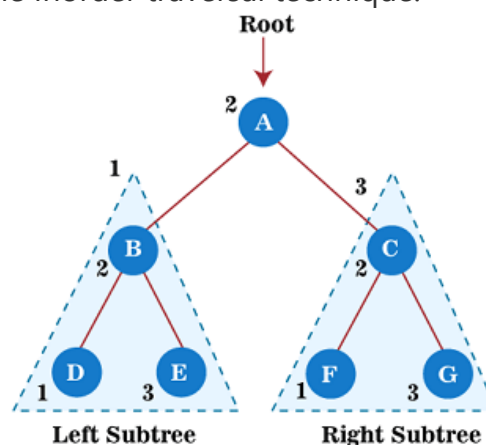
- It is used to get the BST nodes in increasing order.
- It can also be used to get the prefix expression of an expression tree.

Algorithm

1. Until all nodes of the tree are not visited
2. Step 1 - Traverse the left subtree recursively.
3. Step 2 - Visit the root node.
4. Step 3 - Traverse the right subtree recursively.

Example

Now, let's see the example of the Inorder traversal technique.



Now, start applying the inorder traversal on the above tree. First, we traverse the left subtree **B** that will be traversed in inorder. After that, we will traverse the root node **A**. And finally, the right subtree **C** is traversed in inorder.

So, for left subtree **B**, first, its left subtree **D** is traversed. Since node **D** does not have any children, so after traversing it, node **B** will be traversed, and at last, right subtree of node B, that is **E**, is traversed. Node E also does not have any children; therefore, the traversal of the left subtree of root node A is completed.

After that, traverse the root node of a given tree, i.e., **A**.

At last, move towards the right subtree of root node A that is C. So, for right subtree C; first, its left subtree **F** is traversed. Since node **F** does not have any children, node **C** will be traversed, and at last, a right subtree of node C, that is, **G**, is traversed. Node G also does not have any children; therefore, the traversal of the right subtree of root node A is completed.

As all the nodes of the tree are traversed, the inorder traversal of the given tree is completed. The output of the inorder traversal of the above tree is -

D → B → E → A → F → C → G

```
// Java program for inorder traversals
import java.util.*;

// Structure of a Binary Tree Node
class Node {
    int data;
    Node left, right;

    Node(int v)
    {
        data = v;
        left = right = null;
    }
}

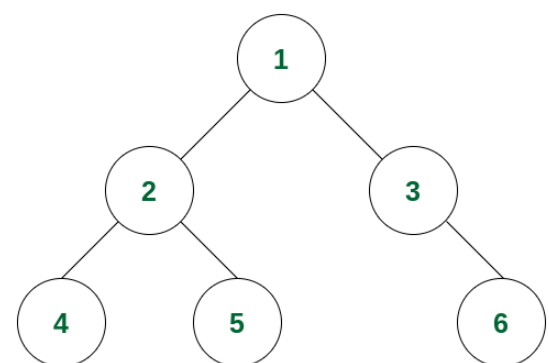
// Main class
class GFG {

    // Function to print inorder traversal
    public static void printInorder(Node node)
    {
        if (node == null)
            return;

        // First recur on left subtree
        printInorder(node.left);

        // Now deal with the node
        System.out.print(node.data + " ");

        // Then recur on right subtree
```



Binary Tree to be traversed


```

        printInorder(node.right);
    }

    // Driver code
    public static void main(String[] args)
    {
        Node root = new Node(1);
        root.left = new Node(2);
        root.right = new Node(3);
        root.left.left = new Node(4);
        root.left.right = new Node(5);
        root.right.right = new Node(6);

        // Function call
        System.out.println(
            "Inorder traversal of binary tree is: ");
        printInorder(root);
    }
}

```

Complexity of Tree traversal techniques

The time complexity of tree traversal techniques discussed above is **$O(n)$** , where '**n**' is the size of binary tree.

Whereas the space complexity of tree traversal techniques discussed above is **$O(1)$** if we do not consider the stack size for function calls. Otherwise, the space complexity of these techniques is **$O(h)$** , where '**h**' is the tree's height.

Types of Binary Tree

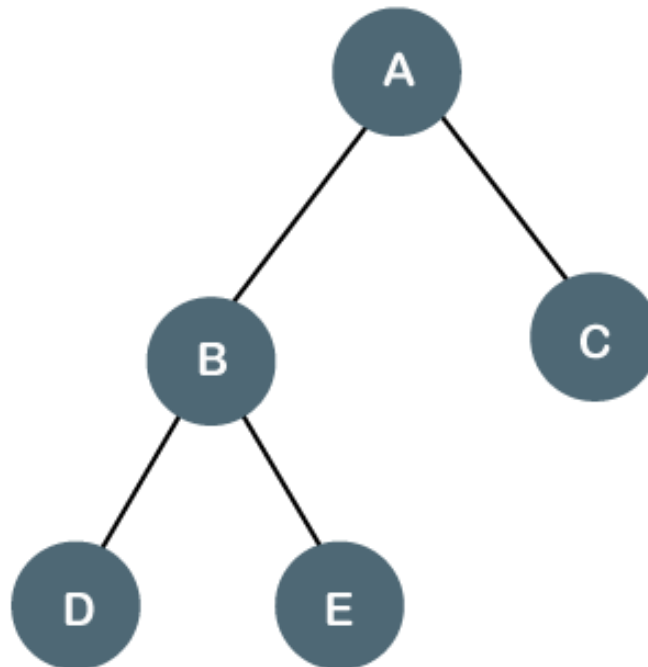
There are five types of Binary tree:

- Full/ proper/ strict Binary tree
- Complete Binary tree
- Perfect Binary tree
- Degenerate Binary tree
- Balanced Binary tree

1. Full/ proper/ strict Binary tree

The full binary tree is also known as a strict binary tree. The tree can only be considered as the full binary tree if each node must contain either 0 or 2 children. The full binary tree can also be defined as the tree in which each node must contain 2 children except the leaf nodes.

Let's look at the simple example of the Full Binary tree.



In the above tree, we can observe that each node is either containing zero or two children; therefore, it is a Full Binary tree.

Properties of Full Binary Tree

- The number of leaf nodes is equal to the number of internal nodes plus 1. In the above example, the number of internal nodes is 5; therefore, the number of leaf nodes is equal to 6.
- The maximum number of nodes is the same as the number of nodes in the binary tree, i.e., $2^{h+1} - 1$.
- The minimum number of nodes in the full binary tree is $2^h - 1$.
- The minimum height of the full binary tree is $\log_2(n+1) - 1$.
- The maximum height of the full binary tree can be computed as:

$$n = 2^h - 1$$

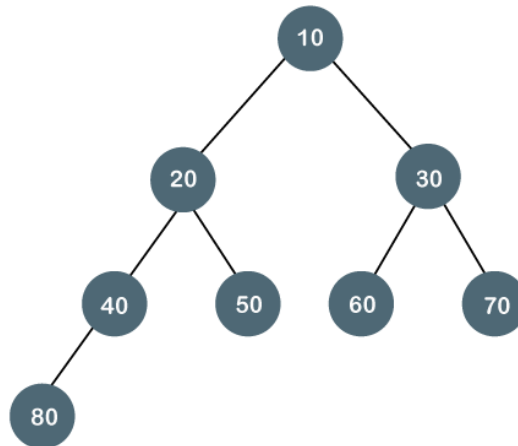
$$n + 1 = 2^h$$

$$h = \frac{n+1}{2}$$

2.Complete Binary Tree

The complete binary tree is a tree in which all the nodes are completely filled except the last level. In the last level, all the nodes must be as left as possible. In a complete binary tree, the nodes should be added from the left.

Let's create a complete binary tree.



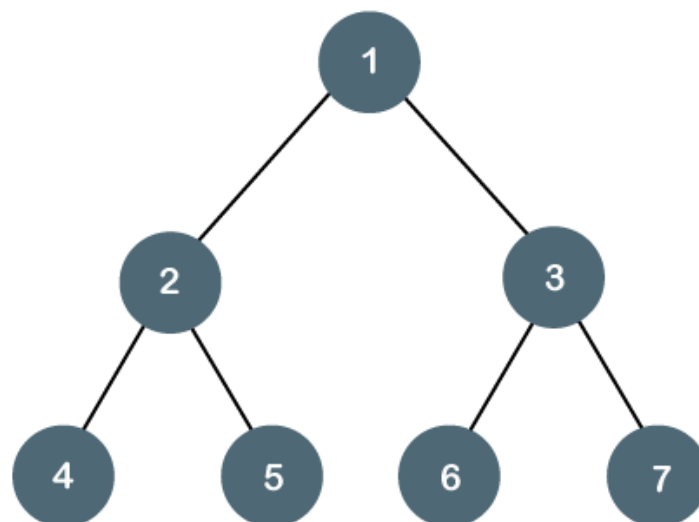
The above tree is a complete binary tree because all the nodes are completely filled, and all the nodes in the last level are added at the left first.

Properties of Complete Binary Tree

- The maximum number of nodes in complete binary tree is $2^{h+1} - 1$.
- The minimum number of nodes in complete binary tree is 2^h .
- The minimum height of a complete binary tree is $\log_2(n+1) - 1$.
- The maximum height of a complete binary tree is

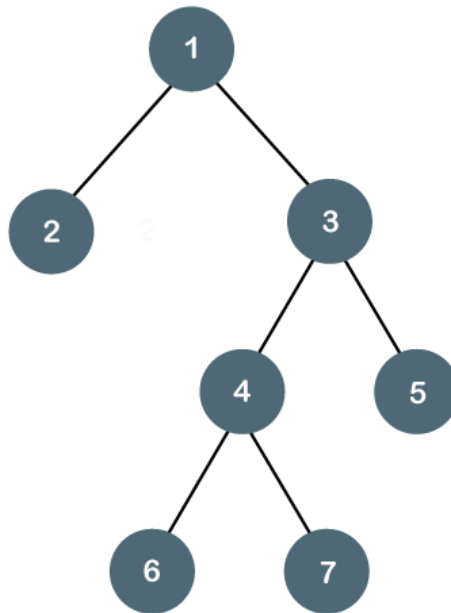
3.Perfect Binary Tree

A tree is a perfect binary tree if all the internal nodes have 2 children, and all the leaf nodes are at the same level.



Let's look at a simple example of a perfect binary tree.

The below tree is not a perfect binary tree because all the leaf nodes are not at the same level.

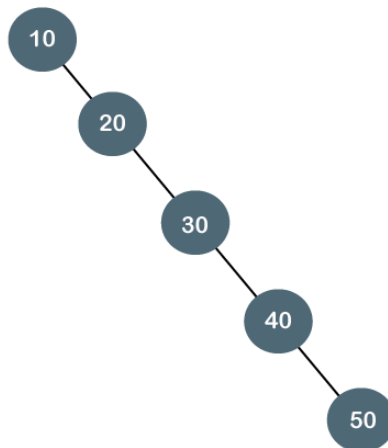


Note: All the perfect binary trees are the complete binary trees as well as the full binary tree, but vice versa is not true, i.e., all complete binary trees and full binary trees are the perfect binary trees.

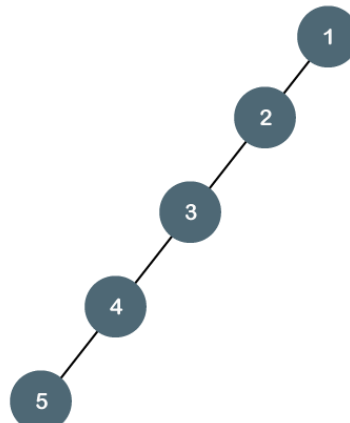
4.Degenerate Binary Tree

The degenerate binary tree is a tree in which all the internal nodes have only one children.

Let's understand the Degenerate binary tree through examples.



The above tree is a degenerate binary tree because all the nodes have only one child. It is also known as a right-skewed tree as all the nodes have a right child only.

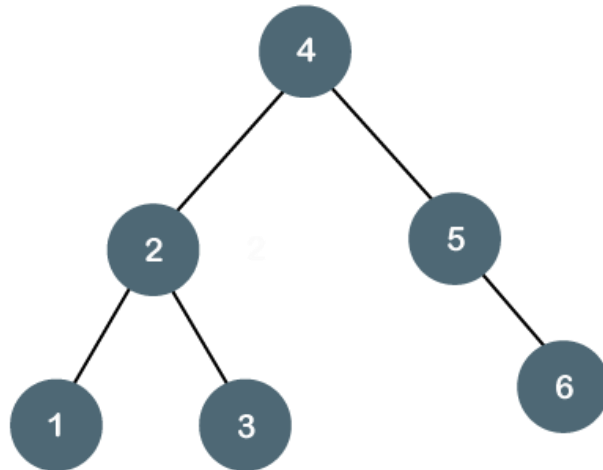


The above tree is also a degenerate binary tree because all the nodes have only one child. It is also known as a left-skewed tree as all the nodes have a left child only

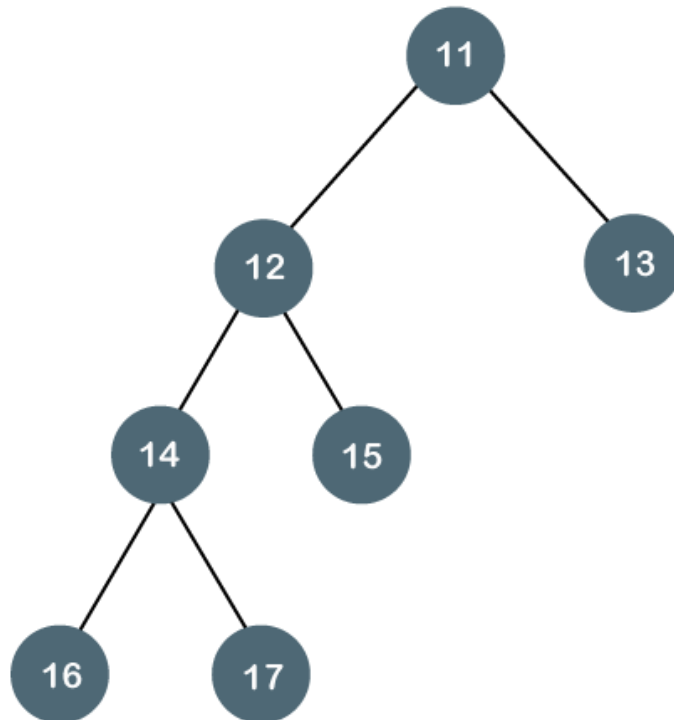
5. Balanced Binary Tree

The balanced binary tree is a tree in which both the left and right trees differ by at most 1. For example, **AVL** and **Red-Black trees** are balanced binary tree.

Let's understand the balanced binary tree through examples.



The above tree is a balanced binary tree because the difference between the left subtree and right subtree is zero.



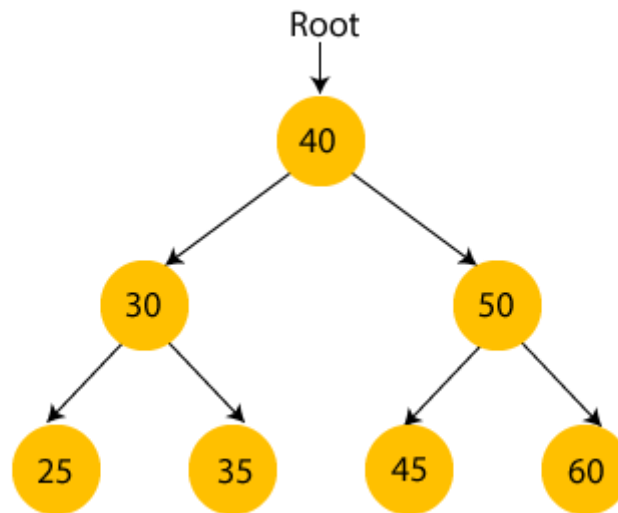
The above tree is not a balanced binary tree because the difference between the left subtree and the right subtree is greater than 1.

Binary Search Tree

What is a Binary Search tree?

A binary search tree follows some order to arrange the elements. In a Binary search tree, the value of left node must be smaller than the parent node, and the value of right node must be greater than the parent node. This rule is applied recursively to the left and right subtrees of the root.

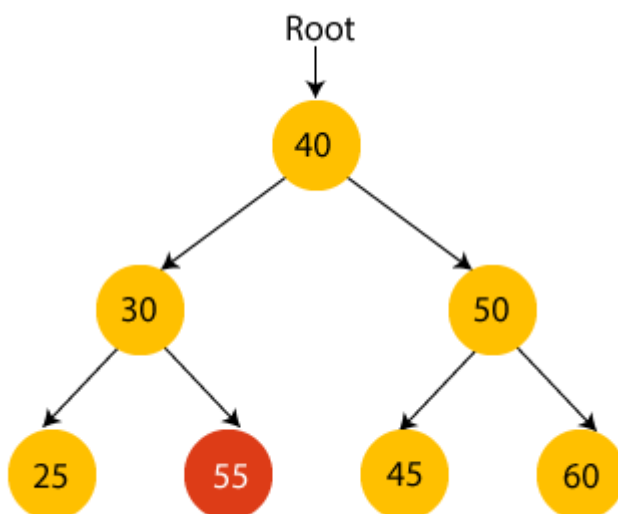
Let's understand the concept of Binary search tree with an example.



In the above figure, we can observe that the root node is 40, and all the nodes of the left subtree are smaller than the root node, and all the nodes of the right subtree are greater than the root node.

Similarly, we can see the left child of root node is greater than its left child and smaller than its right child. So, it also satisfies the property of binary search tree. Therefore, we can say that the tree in the above image is a binary search tree.

Suppose if we change the value of node 35 to 55 in the above tree, check whether the tree will be binary search tree or not.



In the above tree, the value of root node is 40, which is greater than its left child 30 but smaller than right child of 30, i.e., 55. So, the above tree does not satisfy the property of Binary search tree. Therefore, the above tree is not a binary search tree.

Advantages of Binary search tree

- Searching an element in the Binary search tree is easy as we always have a hint that which subtree has the desired element.
- As compared to array and linked lists, insertion and deletion operations are faster in BST.

Example of creating a binary search tree

Now, let's see the creation of binary search tree using an example.

Suppose the data elements are - **45, 15, 79, 90, 10, 55, 12, 20, 50**

- First, we have to insert **45** into the tree as the root of the tree.
- Then, read the next element; if it is smaller than the root node, insert it as the root of the left subtree, and move to the next element.
- Otherwise, if the element is larger than the root node, then insert it as the root of the right subtree.

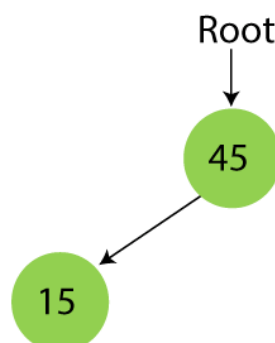
Now, let's see the process of creating the Binary search tree using the given data element. The process of creating the BST is shown below -

Step 1 - Insert 45.



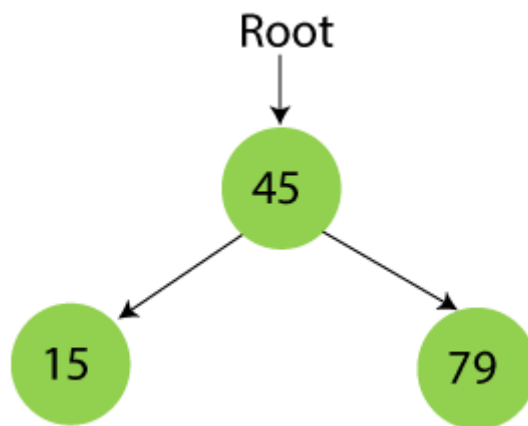
Step 2 - Insert 15.

As 15 is smaller than 45, so insert it as the root node of the left subtree.



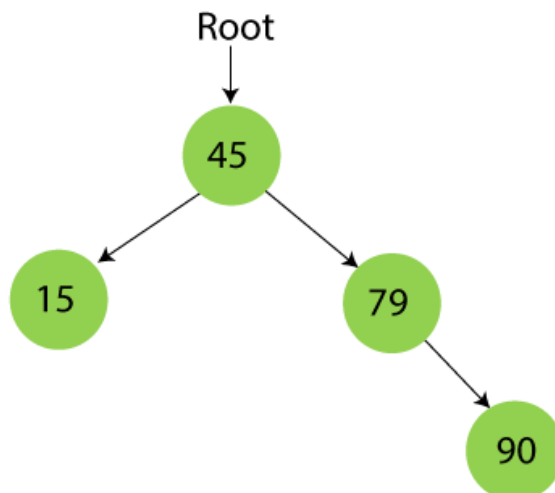
Step 3 - Insert 79.

As 79 is greater than 45, so insert it as the root node of the right subtree.



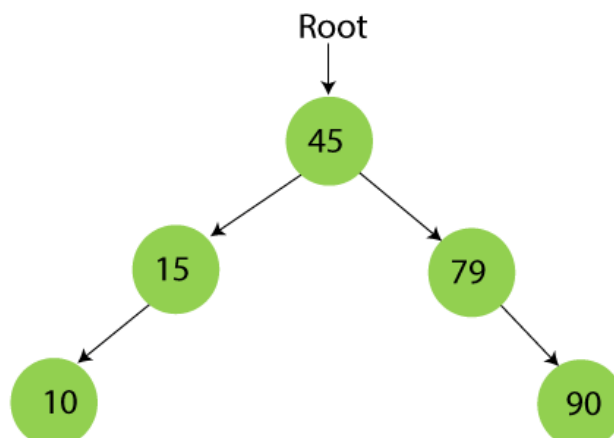
Step 4 - Insert 90.

90 is greater than 45 and 79, so it will be inserted as the right subtree of 79.



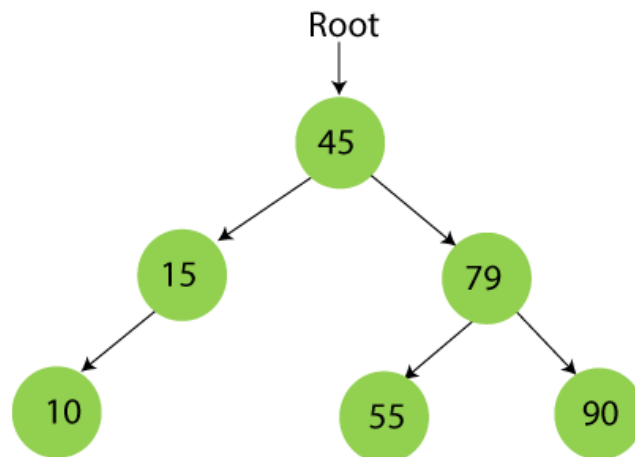
Step 5 - Insert 10.

10 is smaller than 45 and 15, so it will be inserted as a left subtree of 15.



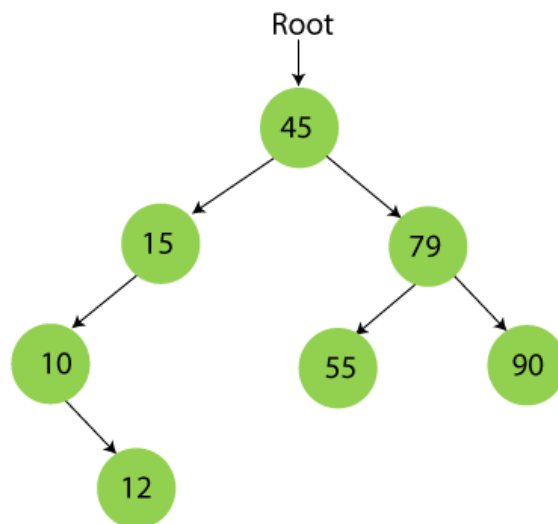
Step 6 - Insert 55.

55 is larger than 45 and smaller than 79, so it will be inserted as the left subtree of 79.



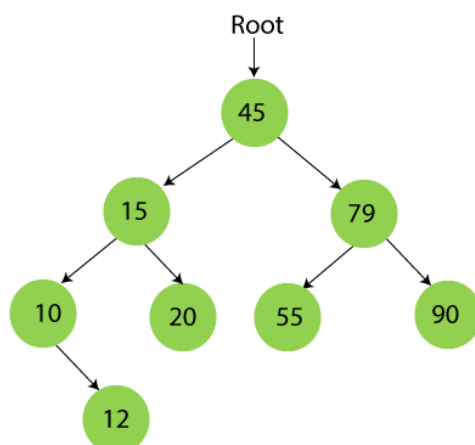
Step 7 - Insert 12.

12 is smaller than 45 and 15 but greater than 10, so it will be inserted as the right subtree of 10.



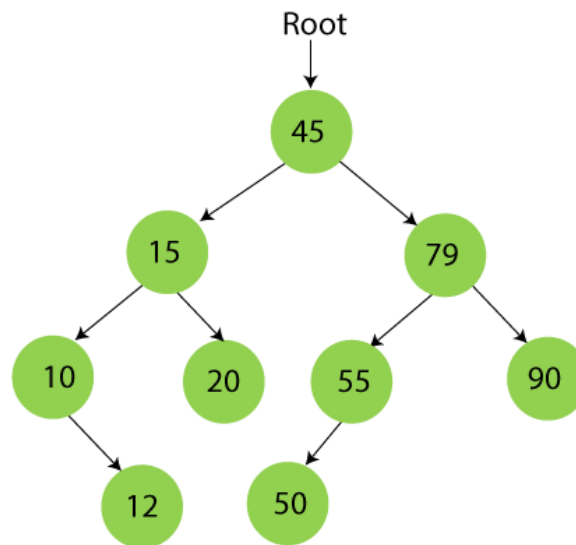
Step 8 - Insert 20.

20 is smaller than 45 but greater than 15, so it will be inserted as the right subtree of 15.



Step 9 - Insert 50.

50 is greater than 45 but smaller than 79 and 55. So, it will be inserted as a left subtree of 55.



Now, the creation of binary search tree is completed. After that, let's move towards the operations that can be performed on Binary search tree.

We can perform insert, delete and search operations on the binary search tree.

Let's understand how a search is performed on a binary search tree.

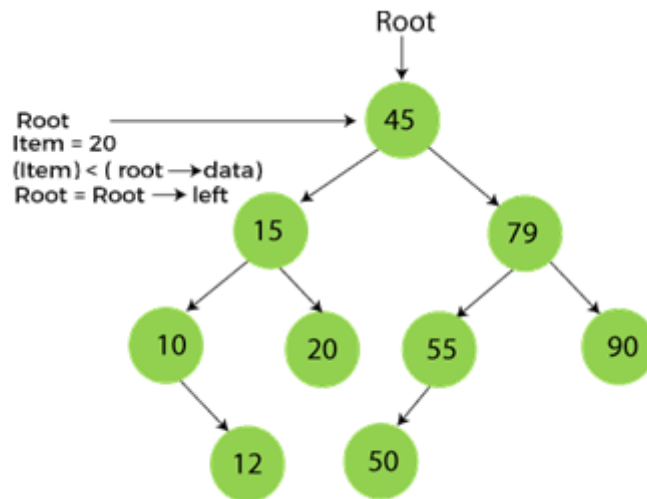
Searching in Binary search tree

Searching means to find or locate a specific element or node in a data structure. In Binary search tree, searching a node is easy because elements in BST are stored in a specific order. The steps of searching a node in Binary Search tree are listed as follows -

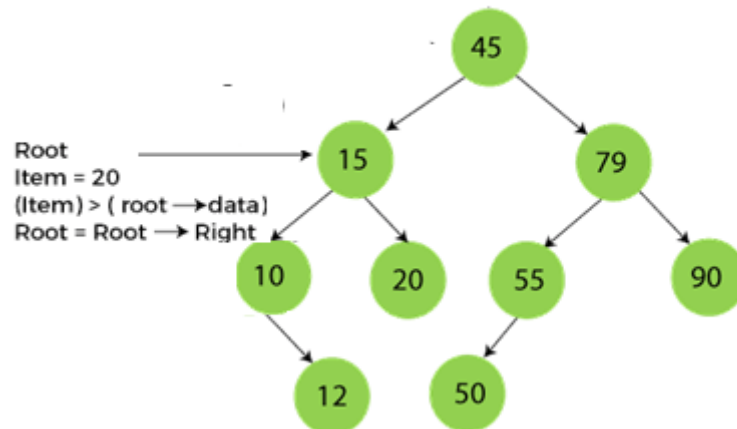
1. First, compare the element to be searched with the root element of the tree.
2. If root is matched with the target element, then return the node's location.
3. If it is not matched, then check whether the item is less than the root element, if it is smaller than the root element, then move to the left subtree.
4. If it is larger than the root element, then move to the right subtree.
5. Repeat the above procedure recursively until the match is found.
6. If the element is not found or not present in the tree, then return NULL.

Now, let's understand the searching in binary tree using an example. We are taking the binary search tree formed above. Suppose we have to find node 20 from the below tree.

Step1:



Step2:



Step3:



Now, let's see the algorithm to search an element in the Binary search tree.

Algorithm to search an element in Binary search tree

1. Search (root, item)
2. Step 1 - if (item = root → data) or (root = NULL)
3. return root
4. else if (item < root → data)
5. return Search(root → left, item)
6. else
7. return Search(root → right, item)
8. END if
9. Step 2 - END

Now let's understand how the deletion is performed on a binary search tree. We will also see an example to delete an element from the given tree.

Deletion in Binary Search tree

In a binary search tree, we must delete a node from the tree by keeping in mind that the property of BST is not violated. To delete a node from BST, there are three possible situations occur -

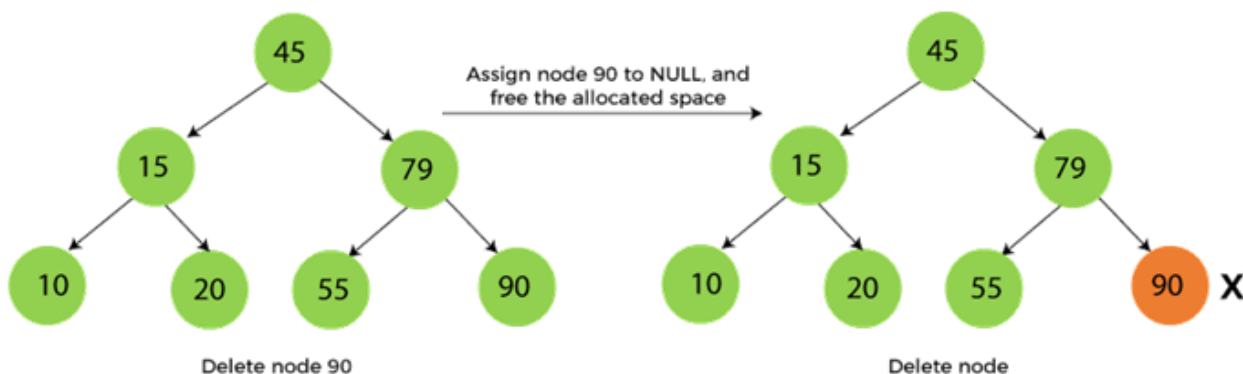
- The node to be deleted is the leaf node, or,
- The node to be deleted has only one child, and,
- The node to be deleted has two children

We will understand the situations listed above in detail.

When the node to be deleted is the leaf node

It is the simplest case to delete a node in BST. Here, we have to replace the leaf node with NULL and simply free the allocated space.

We can see the process to delete a leaf node from BST in the below image. In below image, suppose we have to delete node 90, as the node to be deleted is a leaf node, so it will be replaced with NULL, and the allocated space will free.

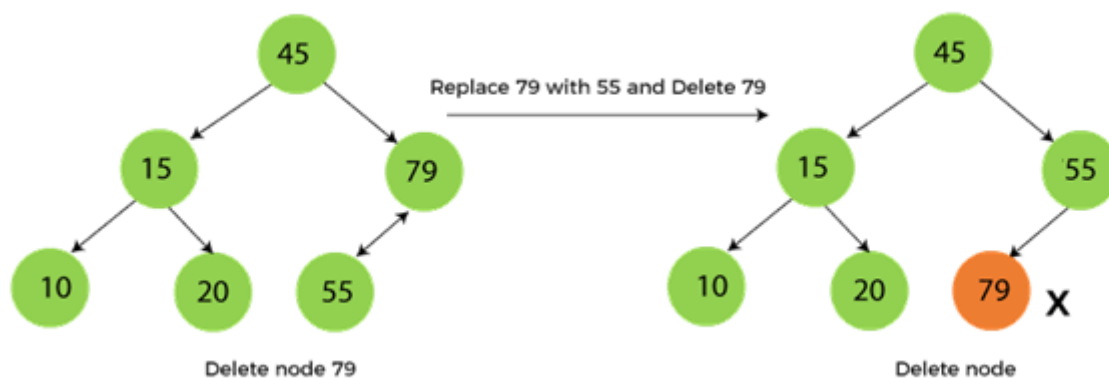


When the node to be deleted has only one child

In this case, we have to replace the target node with its child, and then delete the child node. It means that after replacing the target node with its child node, the child node will now contain the value to be deleted. So, we simply have to replace the child node with NULL and free up the allocated space.

We can see the process of deleting a node with one child from BST in the below image. In the below image, suppose we have to delete the node 79, as the node to be deleted has only one child, so it will be replaced with its child 55.

So, the replaced node 79 will now be a leaf node that can be easily deleted.



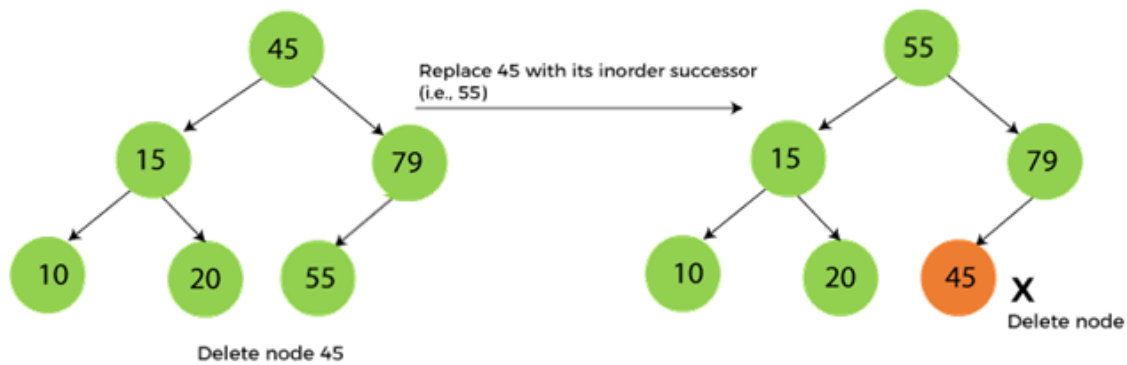
When the node to be deleted has two children

This case of deleting a node in BST is a bit complex among other two cases. In such a case, the steps to be followed are listed as follows -

- First, find the inorder successor of the node to be deleted.
- After that, replace that node with the inorder successor until the target node is placed at the leaf of tree.
- And at last, replace the node with NULL and free up the allocated space.

The inorder successor is required when the right child of the node is not empty. We can obtain the inorder successor by finding the minimum element in the right child of the node.

We can see the process of deleting a node with two children from BST in the below image. In the below image, suppose we have to delete node 45 that is the root node, as the node to be deleted has two children, so it will be replaced with its inorder successor. Now, node 45 will be at the leaf of the tree so that it can be deleted easily.

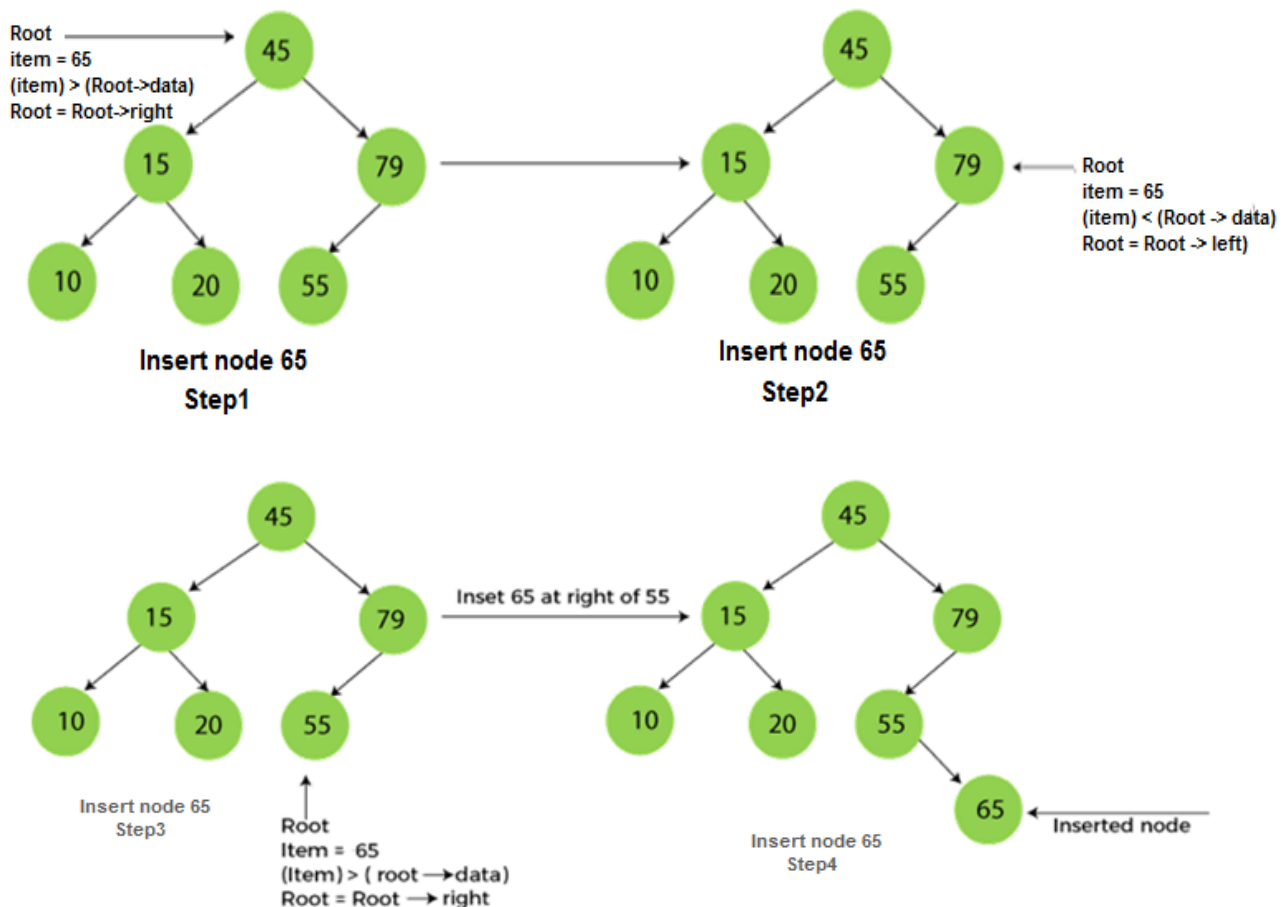


Now let's understand how insertion is performed on a binary search tree.

Insertion in Binary Search tree

A new key in BST is always inserted at the leaf. To insert an element in BST, we have to start searching from the root node; if the node to be inserted is less than the root node, then search for an empty location in the left subtree. Else, search for the empty location in the right subtree and insert the data. Insert in BST is similar to searching, as we always have to maintain the rule that the left subtree is smaller than the root, and right subtree is larger than the root.

Now, let's see the process of inserting a node into BST using an example.



The complexity of the Binary Search tree

Let's see the time and space complexity of the Binary search tree. We will see the time complexity for insertion, deletion, and searching operations in best case, average case, and worst case.

1. Time Complexity

Operations	Best case time complexity	Average case time complexity	Worst case time complexity
Insertion	$O(\log n)$	$O(\log n)$	$O(n)$
Deletion	$O(\log n)$	$O(\log n)$	$O(n)$
Search	$O(\log n)$	$O(\log n)$	$O(n)$

Program to implement search in BST:

```
// Java program to search a given key in a given BST
```

```
class Node {
    int key;
    Node left, right;

    public Node(int item) {
        key = item;
        left = right = null;
    }
}

class BinarySearchTree {
    Node root;

    // Constructor
    BinarySearchTree() {
        root = null;
    }

    // A utility function to insert
    // a new node with given key in BST
    Node insert(Node node, int key) {
        // If the tree is empty, return a new node
        if (node == null) {
            node = new Node(key);
            return node;
        }

        // Otherwise, recur down the tree
        if (key < node.key)
```

```

        node.left = insert(node.left, key);
    else if (key > node.key)
        node.right = insert(node.right, key);

    // Return the (unchanged) node pointer
    return node;
}

// Utility function to search a key in a BST
Node search(Node root, int key) {
    // Base Cases: root is null or key is present at root
    if (root == null || root.key == key)
        return root;

    // Key is greater than root's key
    if (root.key < key)
        return search(root.right, key);

    // Key is smaller than root's key
    return search(root.left, key);
}

// Driver Code
public static void main(String[] args) {
    BinarySearchTree tree = new BinarySearchTree();

    // Inserting nodes
    tree.root = tree.insert(tree.root, 50);
    tree.insert(tree.root, 30);
    tree.insert(tree.root, 20);
    tree.insert(tree.root, 40);
    tree.insert(tree.root, 70);
    tree.insert(tree.root, 60);
    tree.insert(tree.root, 80);

    // Key to be found
    int key = 6;

    // Searching in a BST
    if (tree.search(tree.root, key) == null)
        System.out.println(key + " not found");
    else
        System.out.println(key + " found");

    key = 60;

    // Searching in a BST
    if (tree.search(tree.root, key) == null)
        System.out.println(key + " not found");
    else
        System.out.println(key + " found");
}

```

Output: 6 not found
60 found

AVL Tree Data Structure

AVL Tree is invented by GM Adelson - Velsky and EM Landis in 1962. The tree is named AVL in honour of its inventors.

AVL Tree can be defined as height balanced binary search tree in which each node is associated with a balance factor which is calculated by subtracting the height of its right sub-tree from that of its left sub-tree.

Tree is said to be balanced if balance factor of each node is in between -1 to 1, otherwise, the tree will be unbalanced and need to be balanced.

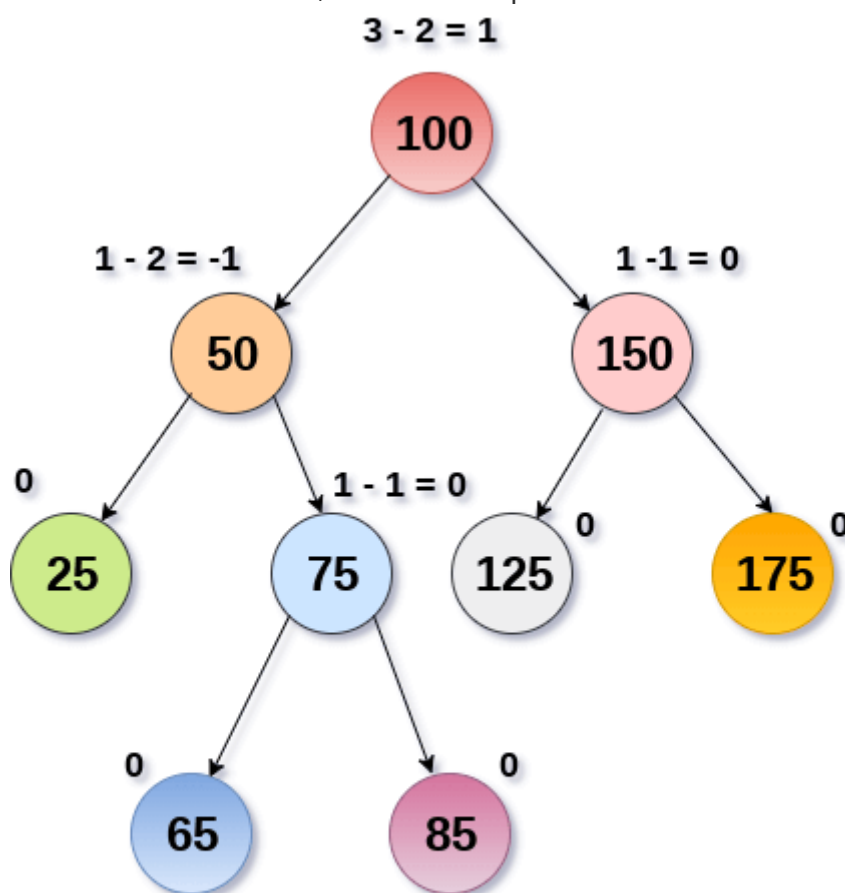
$$\text{Balance Factor (k)} = \text{height (left(k))} - \text{height (right(k))}$$

If balance factor of any node is 1, it means that the left sub-tree is one level higher than the right sub-tree.

If balance factor of any node is 0, it means that the left sub-tree and right sub-tree contain equal height.

If balance factor of any node is -1, it means that the left sub-tree is one level lower than the right sub-tree.

An AVL tree is given in the following figure. We can see that, balance factor associated with each node is in between -1 and +1. therefore, it is an example of AVL tree.



AVL Tree

Complexity

Algorithm	Average case	Worst case
Space	$O(n)$	$O(n)$
Search	$O(\log n)$	$O(\log n)$
Insert	$O(\log n)$	$O(\log n)$
Delete	$O(\log n)$	$O(\log n)$

Operations on AVL tree

Due to the fact that, AVL tree is also a binary search tree therefore, all the operations are performed in the same way as they are performed in a binary search tree. Searching and traversing do not lead to the violation in property of AVL tree. However, insertion and deletion are the operations which can violate this property and therefore, they need to be revisited.

SN	Operation	Description
1	Insertion	Insertion in AVL tree is performed in the same way as it is performed in a binary search tree. However, it may lead to violation in the AVL tree property and therefore the tree may need balancing. The tree can be balanced by applying rotations.
2	Deletion	Deletion can also be performed in the same way as it is performed in a binary search tree. Deletion may also disturb the balance of the tree therefore, various types of rotations are used to rebalance the tree.

Why AVL Tree?

AVL tree controls the height of the binary search tree by not letting it to be skewed. The time taken for all operations in a binary search tree of height h is $O(h)$. However, it can be extended to $O(n)$ if the BST becomes skewed (i.e. worst case). By limiting this height to $\log n$, AVL tree imposes an upper bound on each operation to be $O(\log n)$ where n is the number of nodes.

AVL Rotations

We perform rotation in AVL tree only in case if Balance Factor is other than **-1, 0, and 1**. There are basically four types of rotations which are as follows:

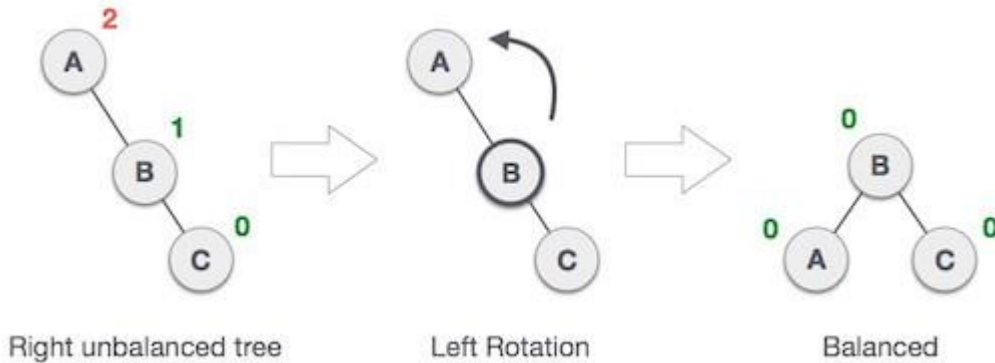
1. L L rotation: Inserted node is in the left subtree of left subtree of A
2. R R rotation : Inserted node is in the right subtree of right subtree of A
3. L R rotation : Inserted node is in the right subtree of left subtree of A
4. R L rotation : Inserted node is in the left subtree of right subtree of A

Where node A is the node whose balance Factor is other than -1, 0, 1.

The first two rotations LL and RR are single rotations and the next two rotations LR and RL are double rotations. For a tree to be unbalanced, minimum height must be at least 2, Let us understand each rotation

1. RR Rotation

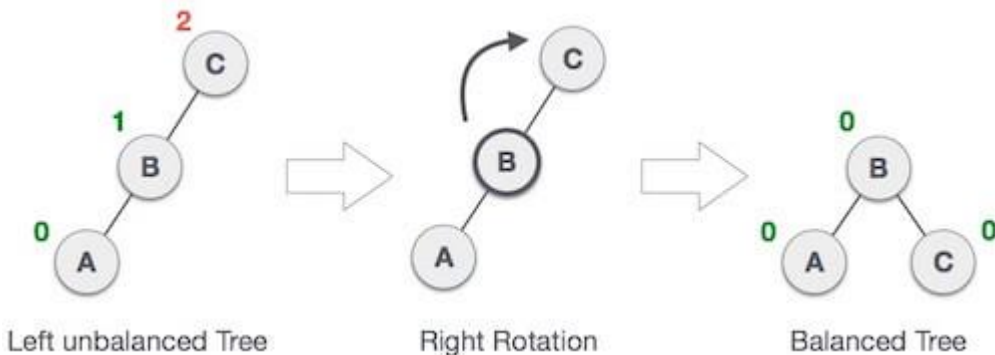
When BST becomes unbalanced, due to a node is inserted into the right subtree of the right subtree of A, then we perform RR rotation, [RR rotation](#) is an anticlockwise rotation, which is applied on the edge below a node having balance factor -2



In above example, node A has balance factor -2 because a node C is inserted in the right subtree of A right subtree. We perform the RR rotation on the edge below A.

2. LL Rotation

When BST becomes unbalanced, due to a node is inserted into the left subtree of the left subtree of C, then we perform LL rotation, [LL rotation](#) is clockwise rotation, which is applied on the edge below a node having balance factor 2.

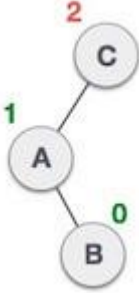
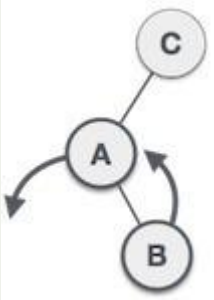
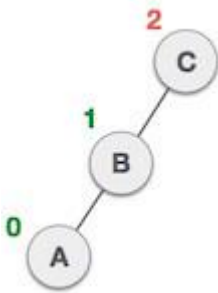
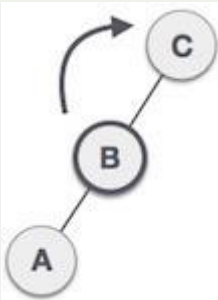
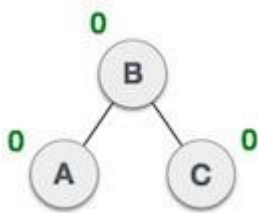


In above example, node C has balance factor 2 because a node A is inserted in the left subtree of C left subtree. We perform the LL rotation on the edge below A.

3. LR Rotation

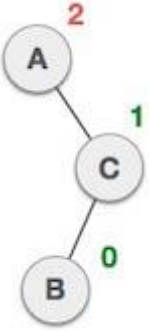
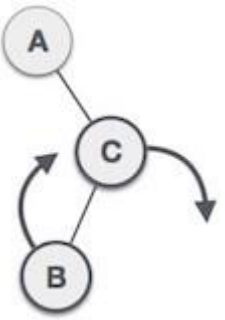
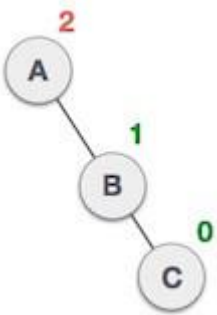
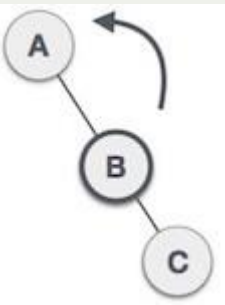
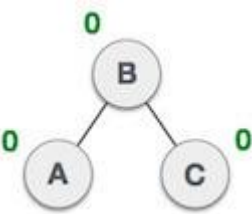
Double rotations are bit tougher than single rotation which has already explained above. LR rotation = RR rotation + LL rotation, i.e., first RR rotation is performed on subtree and then LL rotation is performed on full tree, by full tree we mean the first node from the path of inserted node whose balance factor is other than -1, 0, or 1.

Let us understand each and every step very clearly:

State	Action
	A node B has been inserted into the right subtree of A the left subtree of C, because of which C has become an unbalanced node having balance factor 2. This case is L R rotation where: Inserted node is in the right subtree of left subtree of C
	As LR rotation = RR + LL rotation, hence RR (anticlockwise) on subtree rooted at A is performed first. By doing RR rotation, node A, has become the left subtree of B.
	After performing RR rotation, node C is still unbalanced, i.e., having balance factor 2, as inserted node A is in the left of left of C
	Now we perform LL clockwise rotation on full tree, i.e. on node C. node C has now become the right subtree of node B, A is left subtree of B
	Balance factor of each node is now either -1, 0, or 1, i.e. BST is balanced now.

4. RL Rotation

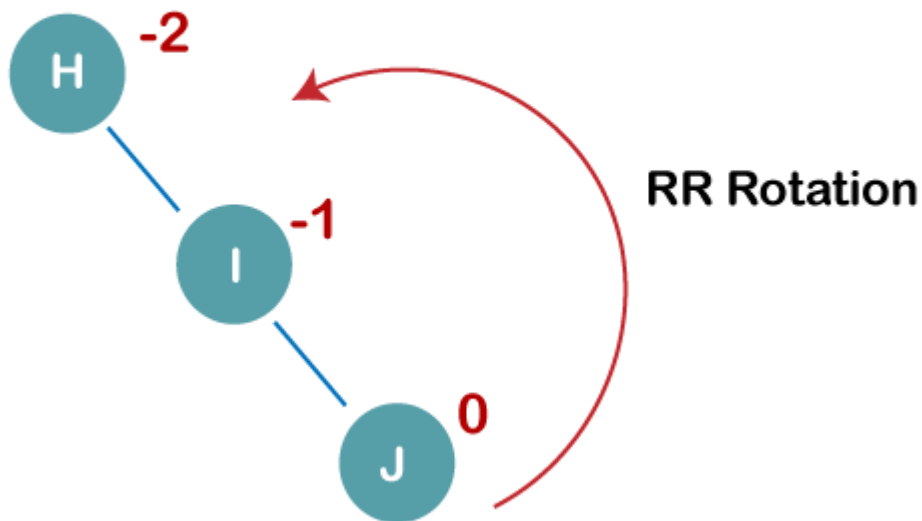
As already discussed, that double rotations are bit tougher than single rotation which has already explained above. RL rotation = LL rotation + RR rotation, i.e., first LL rotation is performed on subtree and then RR rotation is performed on full tree, by full tree we mean the first node from the path of inserted node whose balance factor is other than -1, 0, or 1.

State	Action
	A node B has been inserted into the left subtree of C the right subtree of A, because of which A has become an unbalanced node having balance factor - 2. This case is RL rotation where: Inserted node is in the left subtree of right subtree of A
	As RL rotation = LL rotation + RR rotation, hence, LL (clockwise) on subtree rooted at C is performed first. By doing RR rotation, node C has become the right subtree of B.
	After performing LL rotation, node A is still unbalanced, i.e. having balance factor - 2, which is because of the right-subtree of the right-subtree node A.
	Now we perform RR rotation (anticlockwise rotation) on full tree, i.e. on node A. node C has now become the right subtree of node B, and node A has become the left subtree of B.
	Balance factor of each node is now either -1, 0, or 1, i.e., BST is balanced now.

Q: Construct an AVL tree having the following elements

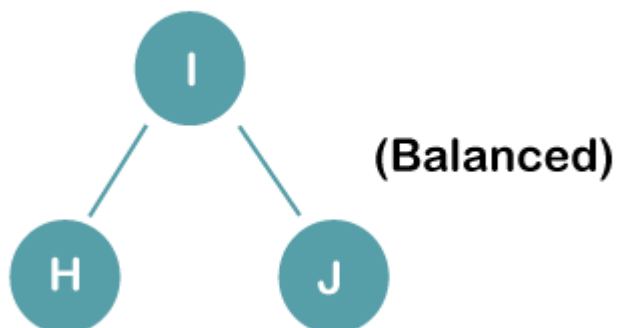
H, I, J, B, A, E, C, F, D, G, K, L

1. Insert H, I, J

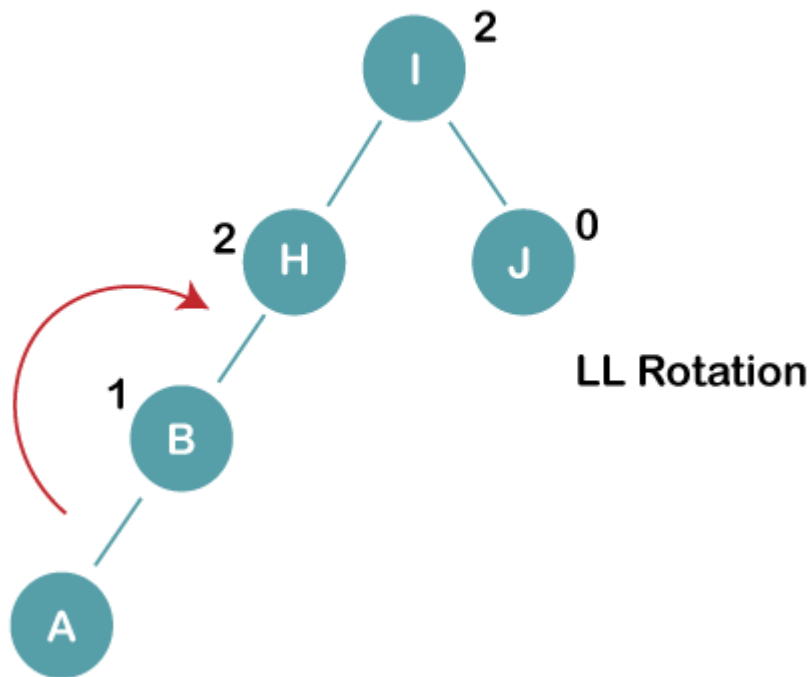


On inserting the above elements, especially in the case of H, the BST becomes unbalanced as the Balance Factor of H is -2. Since the BST is right-skewed, we will perform RR Rotation on node H.

The resultant balance tree is:

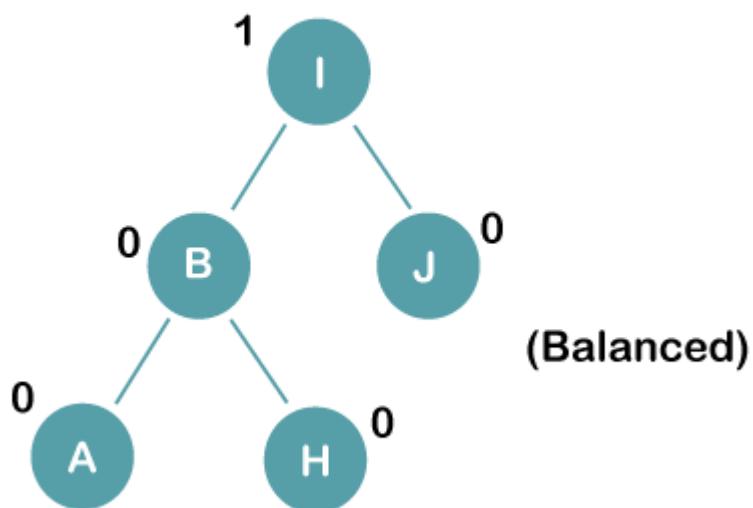


2. Insert B, A

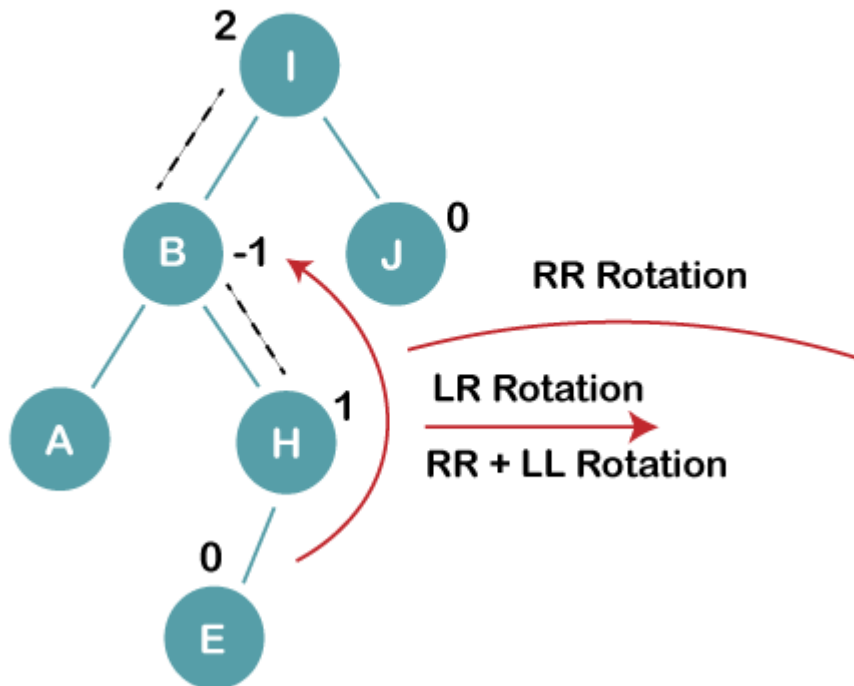


On inserting the above elements, especially in case of A, the BST becomes unbalanced as the Balance Factor of H and I is 2, we consider the first node from the last inserted node i.e. H. Since the BST from H is left-skewed, we will perform LL Rotation on node H.

The resultant balance tree is:



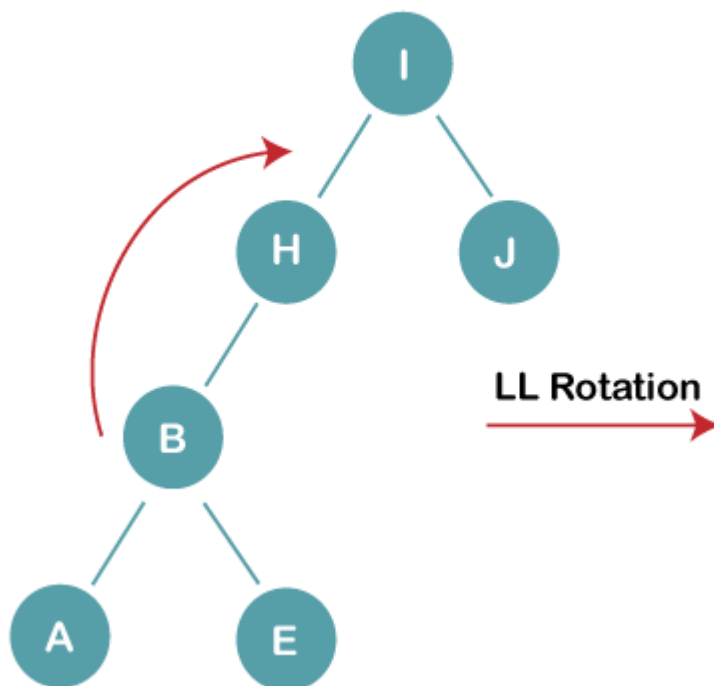
3. Insert E



On inserting E, BST becomes unbalanced as the Balance Factor of I is 2, since if we travel from E to I we find that it is inserted in the left subtree of right subtree of I, we will perform LR Rotation on node I. LR = RR + LL rotation

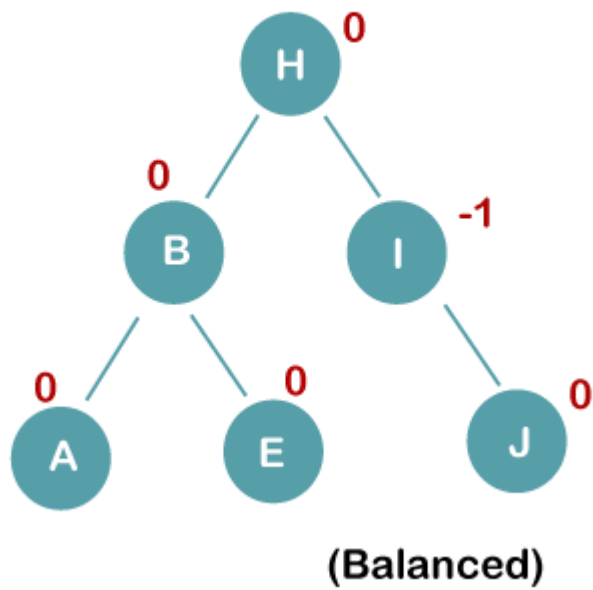
3 a) We first perform RR rotation on node B

The resultant tree after RR rotation is:

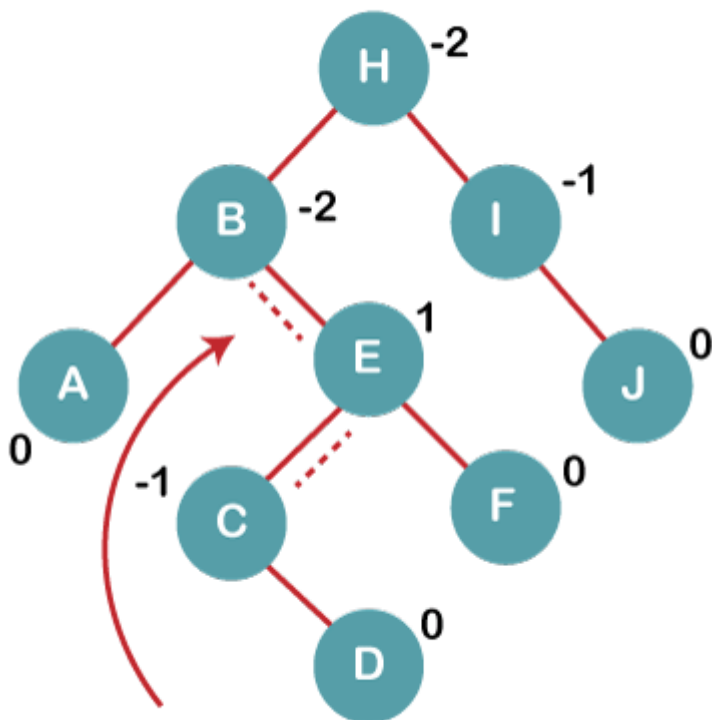


3b) We first perform LL rotation on the node I

The resultant balanced tree after LL rotation is:



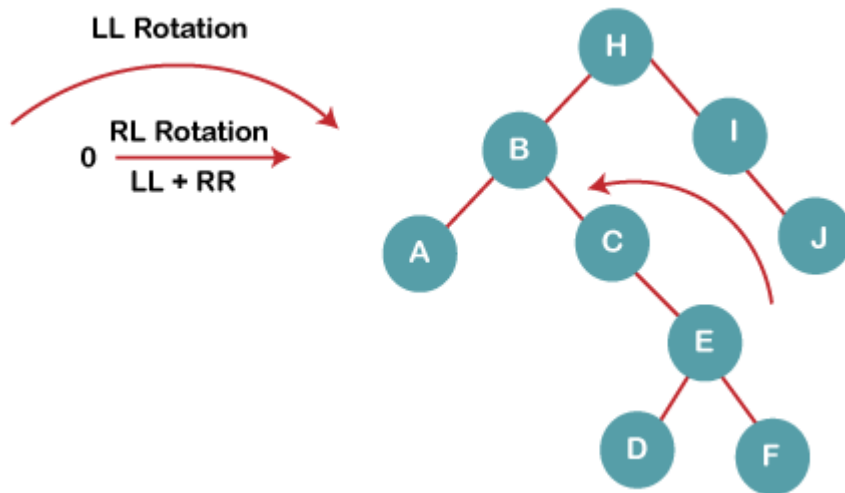
4. Insert C, F, D



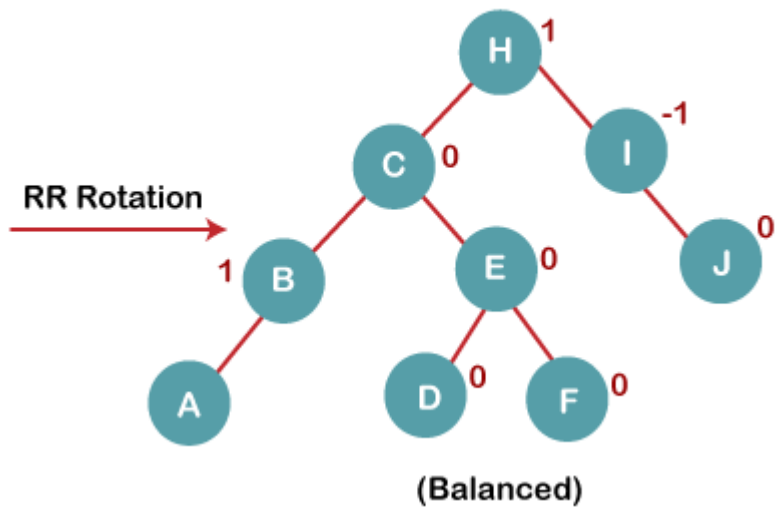
On inserting C, F, D, BST becomes unbalanced as the Balance Factor of B and H is -2, since if we travel from D to B we find that it is inserted in the right subtree of left subtree of B, we will perform RL Rotation on node I. RL = LL + RR rotation.

4a) We first perform LL rotation on node E

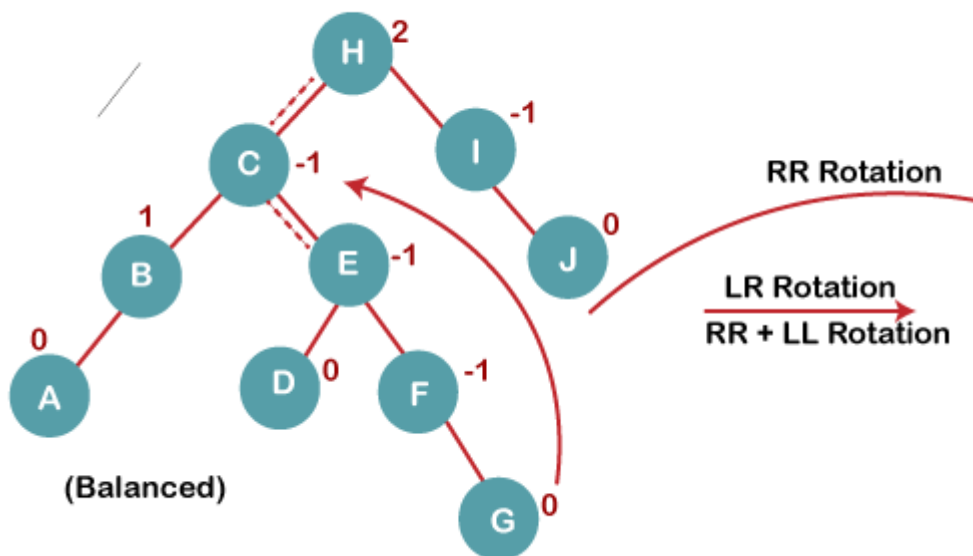
The resultant tree after LL rotation is:



4b) We then perform RR rotation on node B
The resultant balanced tree after RR rotation is:

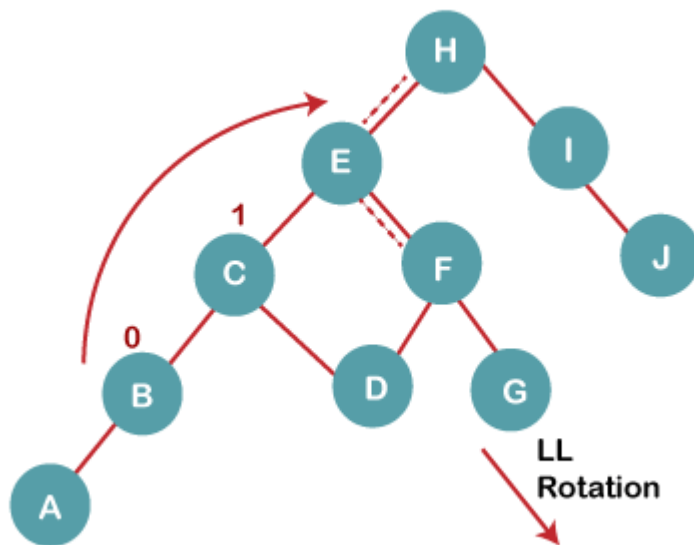


5. Insert G

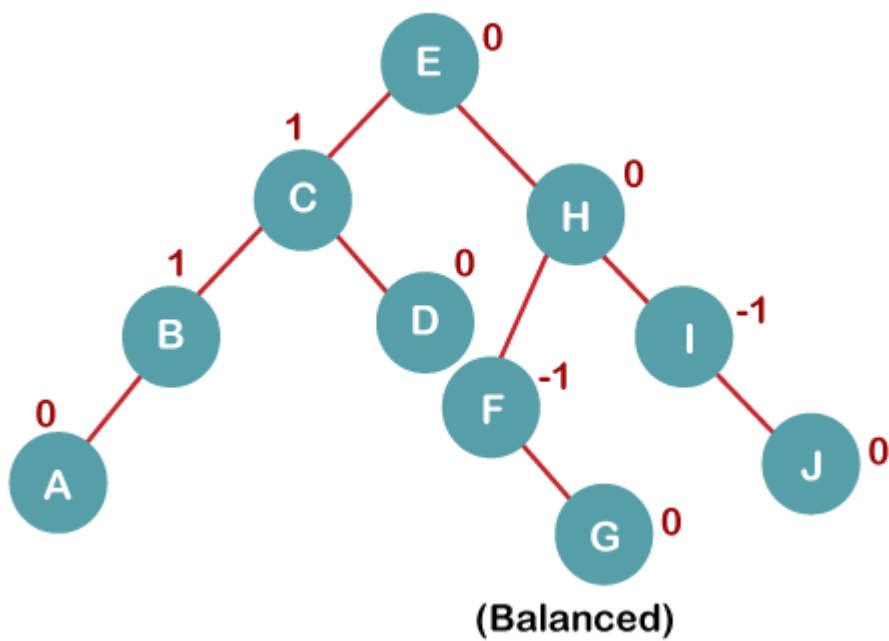


On inserting G, BST become unbalanced as the Balance Factor of H is 2, since if we travel from G to H, we find that it is inserted in the left subtree of right subtree of H, we will perform LR Rotation on node I. LR = RR + LL rotation.

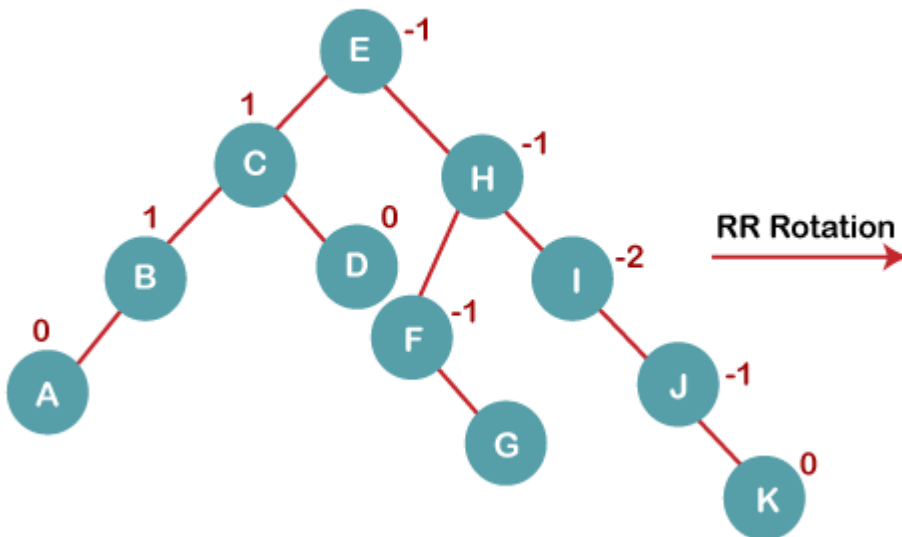
5 a) We first perform RR rotation on node C
The resultant tree after RR rotation is:



5 b) We then perform LL rotation on node H
The resultant balanced tree after LL rotation is:

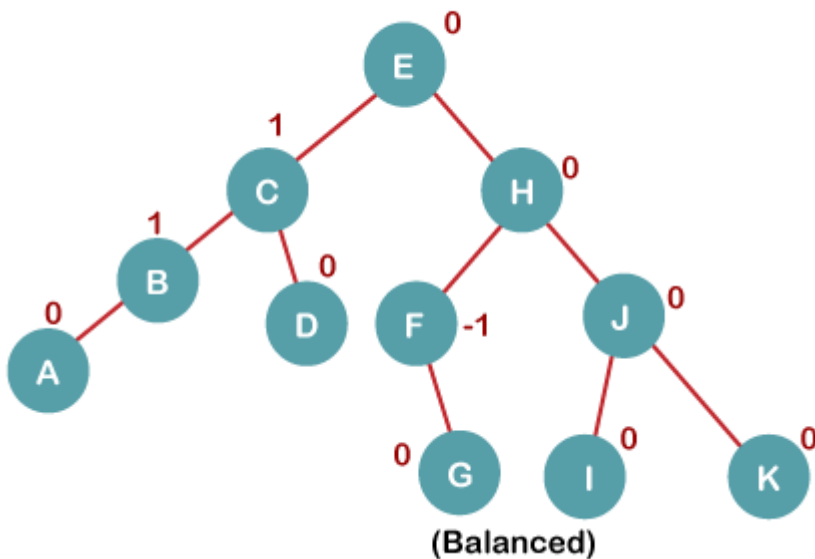


6. Insert K



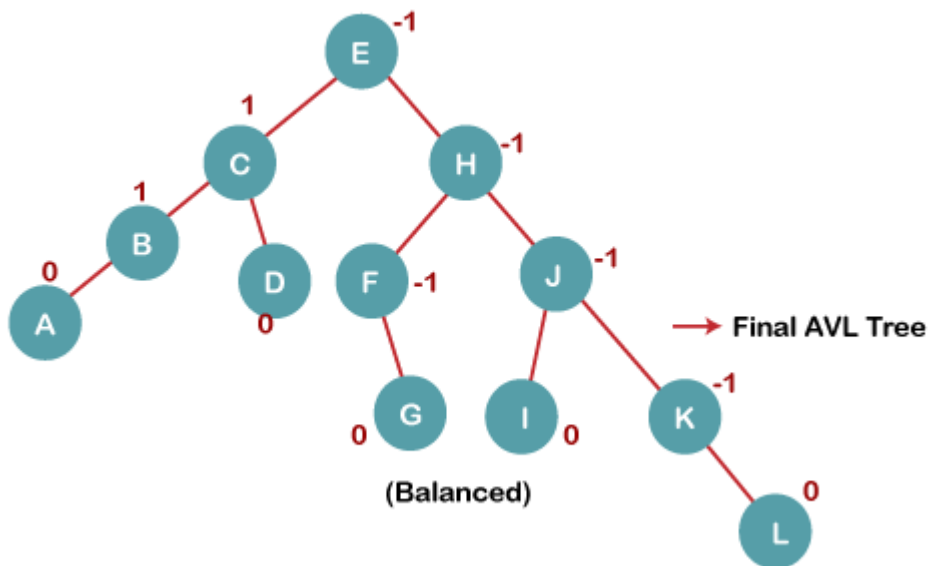
On inserting K, BST becomes unbalanced as the Balance Factor of I is -2. Since the BST is right-skewed from I to K, hence we will perform RR Rotation on the node I.

The resultant balanced tree after RR rotation is:



7. Insert L

On inserting the L tree is still balanced as the Balance Factor of each node is now either, -1, 0, +1. Hence the tree is a Balanced AVL tree



Program to implement insertion in AVL Tree:

```
// Java program for insertion in AVL Tree

class Node {
    int key, height;
    Node left, right;

    Node(int d) {
        key = d;
        height = 1;
    }
}

class AVLTree {
    Node root;

    // A utility function to get the height of the tree
    int height(Node N) {
        if (N == null)
            return 0;

        return N.height;
    }

    // A utility function to get maximum of two integers
    int max(int a, int b) {
        return (a > b) ? a : b;
    }

    // A utility function to right rotate subtree rooted with y
    // See the diagram given above.
    Node rightRotate(Node y) {
        Node x = y.left;
```

```

Node T2 = x.right;

// Perform rotation
x.right = y;
y.left = T2;

// Update heights
y.height = max(height(y.left), height(y.right)) + 1;
x.height = max(height(x.left), height(x.right)) + 1;

// Return new root
return x;
}

// A utility function to left rotate subtree rooted with x
// See the diagram given above.
Node leftRotate(Node x) {
    Node y = x.right;
    Node T2 = y.left;

    // Perform rotation
    y.left = x;
    x.right = T2;

    // Update heights
    x.height = max(height(x.left), height(x.right)) + 1;
    y.height = max(height(y.left), height(y.right)) + 1;

    // Return new root
    return y;
}

// Get Balance factor of node N
int getBalance(Node N) {
    if (N == null)
        return 0;

    return height(N.left) - height(N.right);
}

Node insert(Node node, int key) {

    /* 1. Perform the normal BST insertion */
    if (node == null)
        return (new Node(key));

    if (key < node.key)
        node.left = insert(node.left, key);
    else if (key > node.key)
        node.right = insert(node.right, key);
    else // Duplicate keys not allowed
        return node;
}

```

```

    /* 2. Update height of this ancestor node */
    node.height = 1 + max(height(node.left),
                           height(node.right));

    /* 3. Get the balance factor of this ancestor
       node to check whether this node became
       unbalanced */
    int balance = getBalance(node);

    // If this node becomes unbalanced, then there
    // are 4 cases Left Left Case
    if (balance > 1 && key < node.left.key)
        return rightRotate(node);

    // Right Right Case
    if (balance < -1 && key > node.right.key)
        return leftRotate(node);

    // Left Right Case
    if (balance > 1 && key > node.left.key) {
        node.left = leftRotate(node.left);
        return rightRotate(node);
    }

    // Right Left Case
    if (balance < -1 && key < node.right.key) {
        node.right = rightRotate(node.right);
        return leftRotate(node);
    }

    /* return the (unchanged) node pointer */
    return node;
}

// A utility function to print preorder traversal
// of the tree.
// The function also prints height of every node
void preOrder(Node node) {
    if (node != null) {
        System.out.print(node.key + " ");
        preOrder(node.left);
        preOrder(node.right);
    }
}

public static void main(String[] args) {
    AVLTree tree = new AVLTree();

    /* Constructing tree given in the above figure */
    tree.root = tree.insert(tree.root, 10);
    tree.root = tree.insert(tree.root, 20);
    tree.root = tree.insert(tree.root, 30);
    tree.root = tree.insert(tree.root, 40);

```

```

tree.root = tree.insert(tree.root, 50);
tree.root = tree.insert(tree.root, 25);

/* The constructed AVL Tree would be
      30
     /  \
    20   40
   /  \   \
  10  25   50
 */
System.out.println("Preorder traversal" +
                   " of constructed tree is : ");
tree.preOrder(tree.root);
}
}

```

Output

Preorder traversal of the constructed AVL tree is
 30 20 10 25 40 50

Priority Queue

A priority queue is a **special type of queue** in which each element is associated with a **priority value**. And, elements are served on the basis of their priority. That is, higher priority elements are served first.

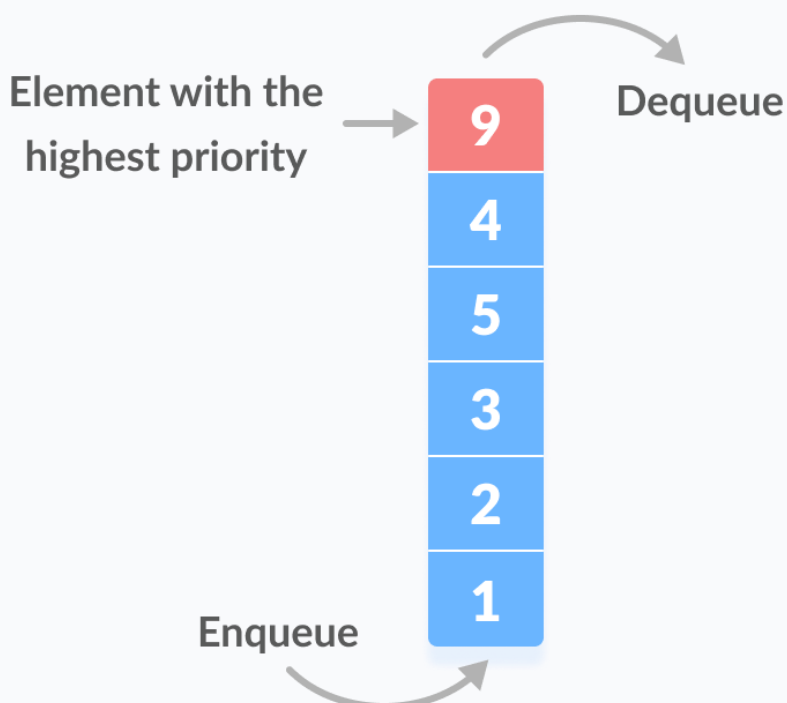
However, if elements with the same priority occur, they are served according to their order in the queue.

Assigning Priority Value

Generally, the value of the element itself is considered for assigning the priority. For example,

The element with the highest value is considered the highest priority element. However, in other cases, we can assume the element with the lowest value as the highest priority element.

We can also set priorities according to our needs.



Removing Highest Priority Element

Difference between Priority Queue and Normal Queue

In a queue, the **first-in-first-out rule** is implemented whereas, in a priority queue, the values are removed **on the basis of priority**. The element with the highest priority is removed first.

Implementation of Priority Queue

Priority queue can be implemented using an array, a linked list, a heap data structure, or a binary search tree. Among these data structures, heap data structure provides an efficient implementation of priority queues.

Hence, we will be using the heap data structure to implement the priority queue in this tutorial. A max-heap is implemented in the following operations. If you want to learn more about it, please visit [max-heap and min-heap](#).

A comparative analysis of different implementations of priority queue is given below.

Operations	peek	insert	delete
Linked List	$O(1)$	$O(n)$	$O(1)$
Binary Heap	$O(1)$	$O(\log n)$	$O(\log n)$
Binary Search Tree	$O(1)$	$O(\log n)$	$O(\log n)$

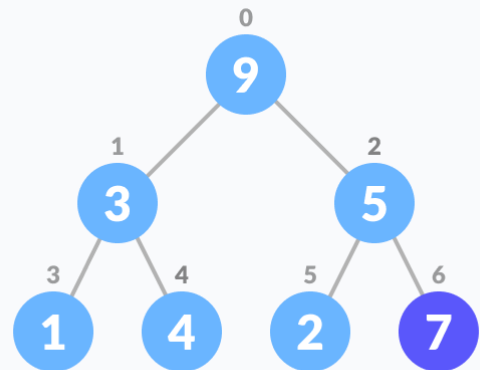
Priority Queue Operations

Basic operations of a priority queue are inserting, removing, and peeking elements.

Before studying the priority queue, please refer to the [heap data structure](#) for a better understanding of binary heap as it is used to implement the priority queue in this article.

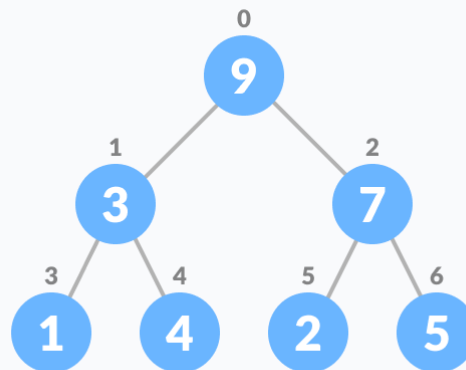
1. Inserting an Element into the Priority Queue

Inserting an element into a priority queue (max-heap) is done by the following steps.



- Insert the new element at the end of the tree.

Insert an element at the end of the queue



- Heapify the tree.

Heapify after insertion

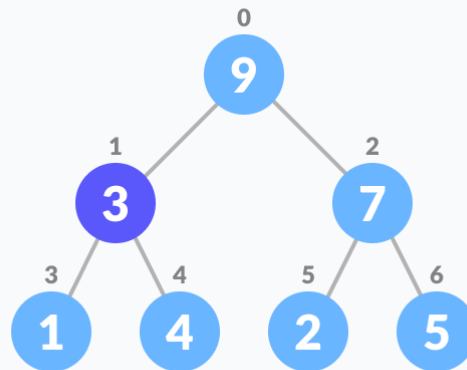
Algorithm for insertion of an element into priority queue (max-heap)

```
If there is no node,  
    create a newNode.  
else (a node is already present)  
    insert the newNode at the end (last node from left to right.)  
  
heapify the array
```

For Min Heap, the above algorithm is modified so that `parentNode` is always smaller than `newNode`.

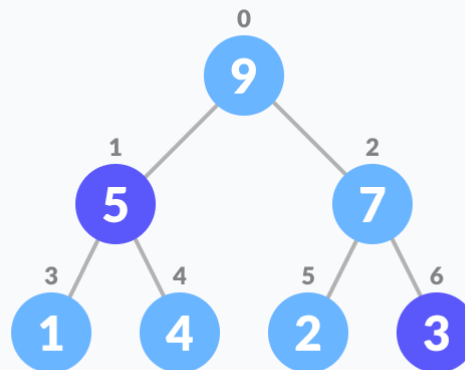
2. Deleting an Element from the Priority Queue

Deleting an element from a priority queue (max-heap) is done as follows:



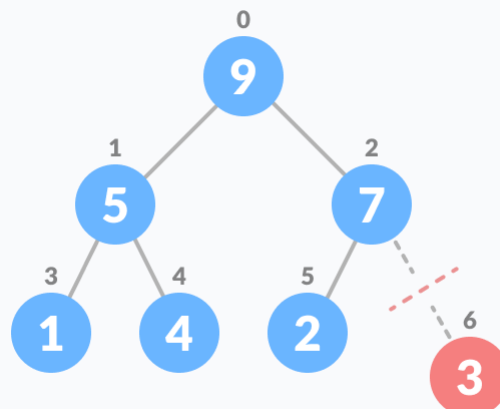
- Select the element to be deleted.
element to be deleted

Select the



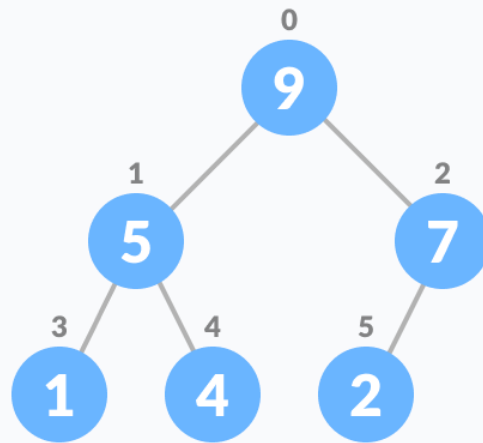
- Swap it with the last element.
last leaf node element

Swap with the



- Remove the last element.
element leaf

Remove the last



- Heapify the tree.

Heapify the priority queue

Algorithm for deletion of an element in the priority queue (max-heap)

```
If nodeToBeDeleted is the leafNode
```

```
    remove the node
```

```
Else swap nodeToBeDeleted with the lastLeafNode
```

```
    remove nodeToBeDeleted
```

```
heapify the array
```

For Min Heap, the above algorithm is modified so that the both `childNodes` are smaller than `currentNode`.

3. Peeking from the Priority Queue (Find max/min)

Peek operation returns the maximum element from Max Heap or minimum element from Min Heap without deleting the node.

For both Max heap and Min Heap

```
return rootNode
```

4. Extract-Max/Min from the Priority Queue

Extract-Max returns the node with maximum value after removing it from a Max Heap whereas Extract-Min returns the node with minimum value after removing it from Min Heap.

Priority Queue Implementations in Python, Java, C, and C++

[Python](#)

[Java](#)

[C](#)

[C++](#)

```
# Priority Queue implementation in Python

# Function to heapify the tree
def heapify(arr, n, i):
    # Find the largest among root, left child and right child
    largest = i
    l = 2 * i + 1
    r = 2 * i + 2

    if l < n and arr[i] < arr[l]:
        largest = l

    if r < n and arr[largest] < arr[r]:
        largest = r

    # Swap and continue heapifying if root is not largest
    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]
        heapify(arr, n, largest)

# Function to insert an element into the tree
def insert(array, newNum):
    size = len(array)
    if size == 0:
        array.append(newNum)
    else:
        array.append(newNum)
        for i in range((size // 2) - 1, -1, -1):
            heapify(array, size, i)
```

```
# Function to delete an element from the tree
def deleteNode(array, num):
    size = len(array)
    i = 0
    for i in range(0, size):
        if num == array[i]:
            break

    array[i], array[size - 1] = array[size - 1], array[i]

    array.remove(size - 1)

    for i in range((len(array) // 2) - 1, -1, -1):
        heapify(array, len(array), i)

arr = []

insert(arr, 3)
insert(arr, 4)
insert(arr, 9)
insert(arr, 5)
insert(arr, 2)

print ("Max-Heap array: " + str(arr))

deleteNode(arr, 4)
print("After deleting an element: " + str(arr))
```

Priority Queue Applications

Some of the applications of a priority queue are:

- Dijkstra's algorithm
- for implementing stack
- for load balancing and interrupt handling in an operating system
- for data compression in Huffman code

Priority Queue using Binary Heap

Priority Queue is an extension of the queue with the following properties:

Every item has a priority associated with it.

An element with high priority is dequeued before an element with low priority.

If two elements have the same priority, they are served according to their order in the queue.

A Binary Heap is a Binary Tree with the following properties:

It is a Complete Tree. This property of Binary Heap makes them suitable to be stored in an array.

A Binary Heap is either Min Heap or Max Heap.

In a Min Binary Heap, the key at the root must be minimum among all keys present in Binary Heap. The same property must be recursively true for all nodes in Binary Tree.

Similarly, in a Max Binary Heap, the key at the root must be maximum among all keys present in Binary Heap. The same property must be recursively true for all nodes in Binary Tree.

Operation on Binary Heap

- **insert(p):** Inserts a new element with priority p.
- **extractMax():** Extracts an element with maximum priority.
- **remove(i):** Removes an element pointed by an iterator i.
- **getMax():** Returns an element with maximum priority.
- **changePriority(i, p):** Changes the priority of an element pointed by i to p.

Implement a priority queue using a min heap in Java:

```
import java.util.ArrayList;
import java.util.List;
```

```
public class MinHeapPriorityQueue<T extends Comparable<T>> {
    private List<T> heap;

    public MinHeapPriorityQueue() {
        heap = new ArrayList<>();
    }

    public void insert(T item) {
        heap.add(item);
        int currentIndex = heap.size() - 1;

        while (currentIndex > 0) {
            int parentIndex = (currentIndex - 1) / 2;
            if (heap.get(currentIndex).compareTo(heap.get(parentIndex)) >= 0) {
                break;
            }

            // Swap the current item with its parent
            T temp = heap.get(currentIndex);
            heap.set(currentIndex, heap.get(parentIndex));
            heap.set(parentIndex, temp);

            currentIndex = parentIndex;
        }
    }

    public T extractMin() {
        if (isEmpty()) {
            throw new IllegalStateException("Priority queue is empty");
        }

        T min = heap.get(0);
        int lastIndex = heap.size() - 1;
        heap.set(0, heap.get(lastIndex));
        heap.remove(lastIndex);

        int currentIndex = 0;
```

```

while (true) {
    int leftChildIndex = 2 * currentIndex + 1;
    int rightChildIndex = 2 * currentIndex + 2;
    int smallestChildIndex = currentIndex;

    if (leftChildIndex < heap.size() &&
heap.get(leftChildIndex).compareTo(heap.get(smallestChildIndex)) < 0) {
        smallestChildIndex = leftChildIndex;
    }

    if (rightChildIndex < heap.size() &&
heap.get(rightChildIndex).compareTo(heap.get(smallestChildIndex)) < 0) {
        smallestChildIndex = rightChildIndex;
    }

    if (smallestChildIndex == currentIndex) {
        break;
    }

    // Swap the current item with the smallest child
    T temp = heap.get(currentIndex);
    heap.set(currentIndex, heap.get(smallestChildIndex));
    heap.set(smallestChildIndex, temp);

    currentIndex = smallestChildIndex;
}

return min;
}

public boolean isEmpty() {
    return heap.isEmpty();
}

public int size() {
    return heap.size();
}

public static void main(String[] args) {
    MinHeapPriorityQueue<Integer> minHeap = new
MinHeapPriorityQueue<>();

```

```
minHeap.insert(5);
minHeap.insert(2);
minHeap.insert(8);
minHeap.insert(1);
minHeap.insert(4);

while (!minHeap.isEmpty()) {
    System.out.println(minHeap.extractMin());
}
}
```